



Savitribai Phule Pune University
(Formerly University of Pune)

T.Y.B.Sc.(Computer Applications)/ T.Y.B.C.A. – Semester-V

(Faculty of Science and Technology)

(For Colleges Affiliated to Savitribai Phule Pune University)

**Choice Based Credit System (CBCS) Syllabus Under
National Education Policy (NEP)**

To be implemented from Academic Year 2026-2027

Title of the Course: B.C.A./ B.Sc. (CA)



Savitribai Phule Pune University
(Formerly University of Pune)

T.Y.B.Sc.(Computer Applications)/ T.Y.B.C.A.

Semester-V

WORKBOOK

Course Code: CA – 305–MJP
Course Title: Fundamentals of Data Science

Student Name			
College Name			
Roll No.		Division	
Exam Seat No.		Academic Year	

From the Chairman's Desk

It gives me great pleasure to present this workbook prepared by the Board of Studies in Computer Applications.

The workbook has been prepared with the objectives of bringing uniformity in the implementation of lab assignments across all affiliated colleges, acting as a ready reference for both fast and slow learners, and facilitating continuous assessment using clearly defined rubrics.

The workbook provides, for each of the assignments, the aims, prerequisites, related theoretical concepts with suitable examples wherever necessary, guidelines for the faculty/lab administrator, instructions for the students to perform assignments, and a set of exercises divided into three sets.

I am thankful to the Chairman of this course and the entire team of editors. I am also thankful to the reviewers and members of BOS, **Dr. A.B. Nimbalkar** and **Dr. Razzak Sayyad**. I thank all members of BOS and everyone who has contributed directly or indirectly to the preparation of the workbook.

Constructive criticism is welcome and should be communicated to the Chairman of the Course and overall coordinator, **Dr. A. B. Nimbalkar**. Affiliated colleges are requested to collect feedback from the students for further improvements.

I am thankful to Hon. Vice Chancellor of Savitribai Phule Pune University **Prof. Dr. Suresh Gosavi**, Pro Vice Chancellor **Dr. Parag Kalkar**, and the Dean of Faculty of Science and Technology **Prof. Dr. Pramod D. Patil** for their support and guidance.

Prof. Dr. S. S. Sane

Chairman, BOS in Computer Applications Savitribai
Phule Pune University, Pune

Editors:

Dr. Reena shinde	Sinhgad College of Science,Pune
Mrs. Savita Bhujbal	PDEA's Annasaheb Magar Mahavidyalaya, Hadapsar, Pune.
Mrs. Smita Ghorpade	KTHM College,Nashik
Mrs. Vidya Shelar	S.M.Joshi College Hadapsar,Pune.
Mrs. Mamta Bendale	K.V.N.Naik Arts,Commerce and Science ,Nashik

Reviewed by:

- **Dr. A. B. Nimbalkar**

Assistant Professor,

PDEA's Waghire College, Pune.

- **Dr. Razzak Sayyad**

Assistant Professor,

Ahmednagar College, Ahilyanagar.

INDEX

Sr. No	Assignment Name	Pg. No.
1	The Data Science Environment	7
2	Statistical Data Analysis	15
3	Data Preprocessing	30
4	Data Visualization	37

Assignment Completion Sheet

CA – 305–MJP Fundamentals of Data Science Assignments Internal Evaluation			
Sr. No.	Assignment Name	Marks Obtained	Instructor Sign
1	The Data Science Environment		
2	Statistical Data Analysis		
3	Data Preprocessing		
4	Data Visualization		

Batch-Incharge

CERTIFICATE

This is to certify that **Mr./Ms.**_____

Exam Seat No. _____ have successfully completed and submitted the coursework for lab course **CA – 305–MJP** on **Fundamentals of Data Science, T.Y.B.Sc.(C.A.)/ TYBCA – Sem. V** prescribed by Savitribai Phule Pune University during Academic year-____ and has scored marks_____ out of _____ Marks.

Batch-Incharge

Head of the Department. /Coordinator

Internal examiner

External examiner

1. About the work book:

This LAB book / workbook is intended to be used by T.Y.B.C.A. / T.Y.B.Sc. (Computer application) students for the **Fundamentals of Data Science** assignments in **Semester–V**. This workbook is designed by considering all the practical concepts / topics mentioned in syllabus. The lab book is to be used as a hands-on resource, reference and record of assignment submission and completion by the student. The lab book contains the set of assignments which the student must complete as a part of this course.

2. The objectives of this LAB-Book are:

- Defining the scope of the course.
- To bring uniformity in the practical conduction and implementation in all colleges affiliated to SPPU.
- To have continuous assessment of the course and students.
- Providing ready reference for the students during practical implementation.
- Provide more options to students so that they can have good practice before facing the examination.
- Catering to the demand of slow and fast learners and accordingly providing the practice assignments to them.

3. Instructions to the students:

- Students are expected to carry this book every time they come to the lab for computer science practical.
- Students should prepare oneself beforehand for the assignment by reading the relevant material.
- Instructor will specify which problems to solve in the lab during the allotted slot and student should complete them and get verified by the instructor.
- However, student should spend additional hours in Lab and at home to cover as many problems as possible given in this work book.

4. Submission of Assignments:

- The Python Programming assignments are to be submitted by the student in the form of a journal containing each assignment sheet.
- Each assignment includes the Assignment Title, Problem statement, Date of submission, Assessment date, Assessment Marks or Grade and Batch In charge's sign.

5. Assessment:

- Continuous assessment of laboratory work is to be done based on overall performance and lab assignments performance of student.
- Each lab assignment assessment will be assigned grade / marks based on parameters with appropriate weightage.
- Suggested parameters for overall assessment as well as each lab assignment assessment includes: timely completion, performance, innovation, efficient codes and good programming practices.
- Students will be assessed for each exercise on a scale from 0 to 5.

Not done	0	Needs improvement	3
Incomplete	1	Complete	4
Late Complete	2	Well Done	5

Assignment 1: The Data Science Environment

Installing Python

Python is a widely used high-level programming language. To begin working with Python, it must first be installed on your system.

Steps to Install Python

1. Visit the official Python website: <https://www.python.org/downloads/>
2. Download the latest version of **Python 3** (recommended version).
3. Choose the appropriate installer based on your operating system:
 - **Windows:** Download the .exe installer
 - **macOS:** Download the .pkg installer
 - **Linux:** Python is usually pre-installed. If not, install it using the system's package manager (e.g., apt, dnf, etc.)
4. Run the installer and follow the instructions on the screen.
 - **Important (Windows users):** Ensure that the option “Add Python to PATH” is selected during installation.

Python Execution Environment

Introduction

A **Python environment** refers to the setup in which Python programs are written, executed, and managed. It includes the Python interpreter, libraries, and tools required to run Python code.

Different environments provide different features:

- Some are **command-line based**, offering direct interaction with the interpreter
- Some are **graphical interfaces**, making it easier for beginners
- Some are **web-based**, allowing execution without local installation

Types of Python Environments

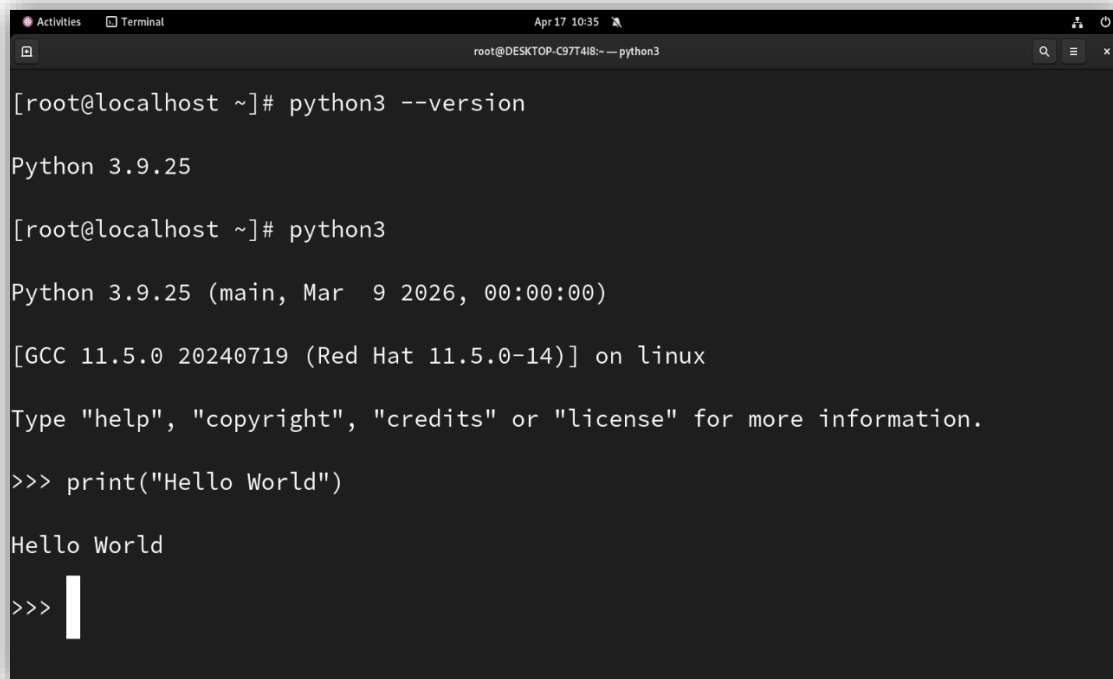
A. Terminal / Shell-Based Environment

- This is the most basic way to run Python programs.

- Users interact directly with the Python interpreter through the command line.
- Suitable for quick testing and system-level scripting.

In Linux (CentOS):

1. Open Terminal
 - Check Python version: `python3 --version`
 - **If not installed**, use the package manager:
 - **For CentOS 7 / 8:** `sudo yum install python3`
2. Start Python interpreter:
 - `python3`

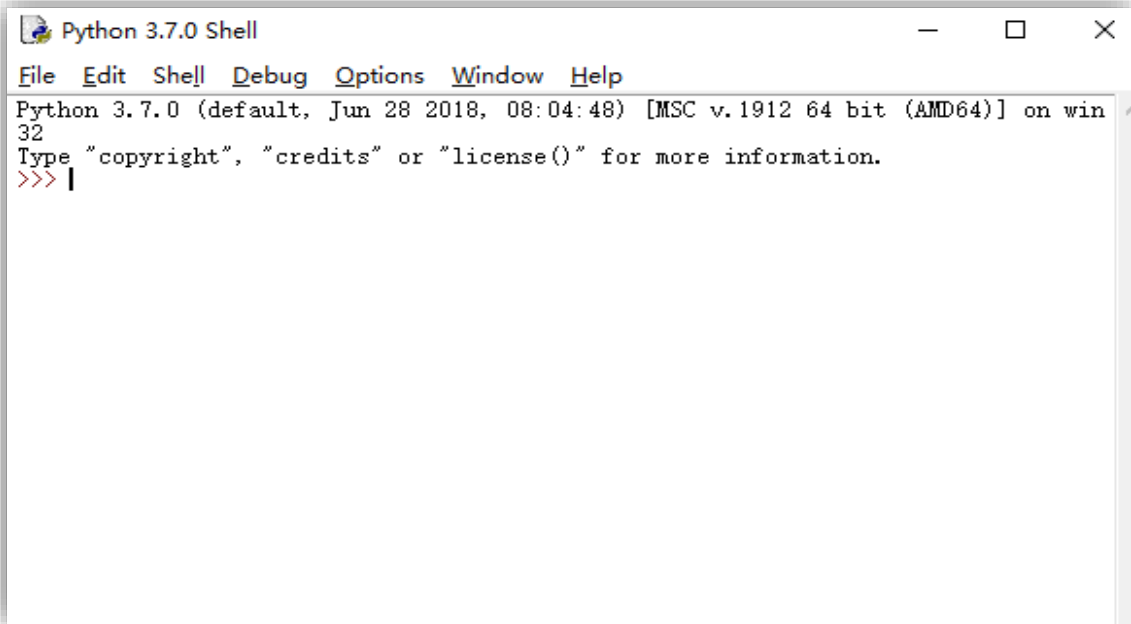
A screenshot of a Linux terminal window. The window title is "Terminal" and the address bar shows "root@DESKTOP-C97T4I8:~ -- python3". The terminal output shows the command `python3 --version` returning `Python 3.9.25`. Then, the command `python3` is entered, starting the Python interpreter. The interpreter displays its version and build information: `Python 3.9.25 (main, Mar 9 2026, 00:00:00)` and `[GCC 11.5.0 20240719 (Red Hat 11.5.0-14)] on linux`. It prompts the user to type "help", "copyright", "credits" or "license" for more information. The user enters `>>> print("Hello World")`, and the interpreter outputs `Hello World`. The prompt `>>>` is shown again with a cursor.

```
[root@localhost ~]# python3 --version
Python 3.9.25

[root@localhost ~]# python3
Python 3.9.25 (main, Mar 9 2026, 00:00:00)
[GCC 11.5.0 20240719 (Red Hat 11.5.0-14)] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> print("Hello World")
Hello World
>>>
```

B. IDLE (Integrated Development and Learning Environment)

- IDLE is the default environment provided with Python.
- It includes a simple text editor and interactive shell.
- Best suited for beginners learning Python basics.



C. Jupyter Notebook

- A web-based interactive environment for writing and running Python code.
- It allows users to write and execute Python code in small blocks (cells).
- Supports live code execution, data visualization, and documentation.
- Widely used in data science, machine learning, and research.

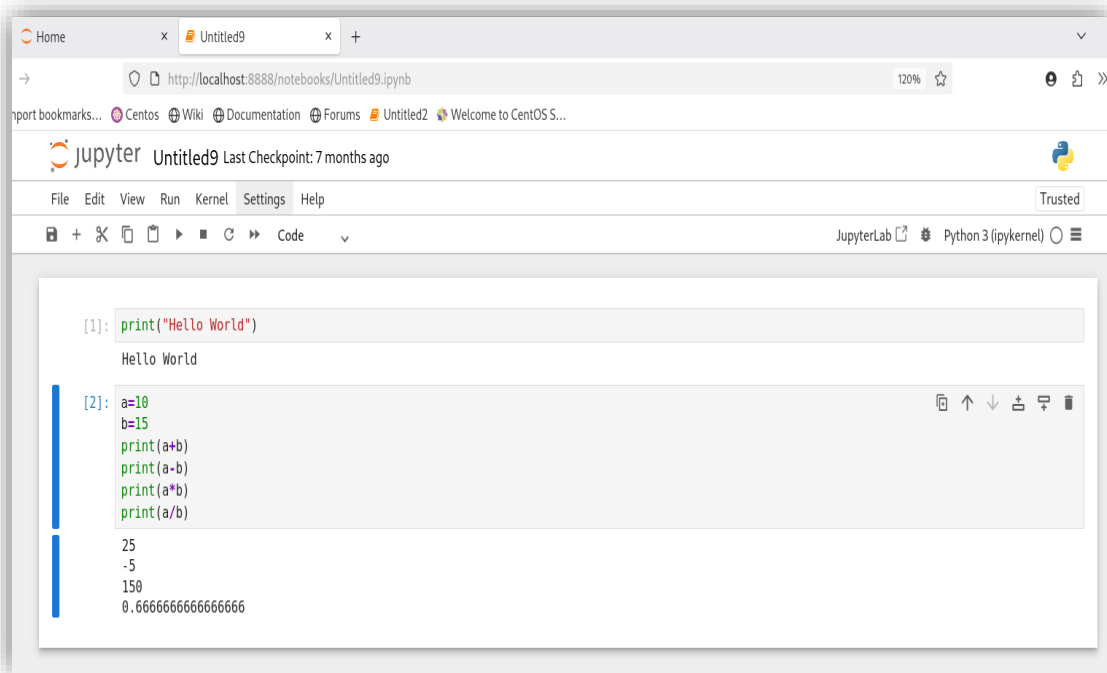
In Linux (CentOS):

Installation:

pip3 install notebook

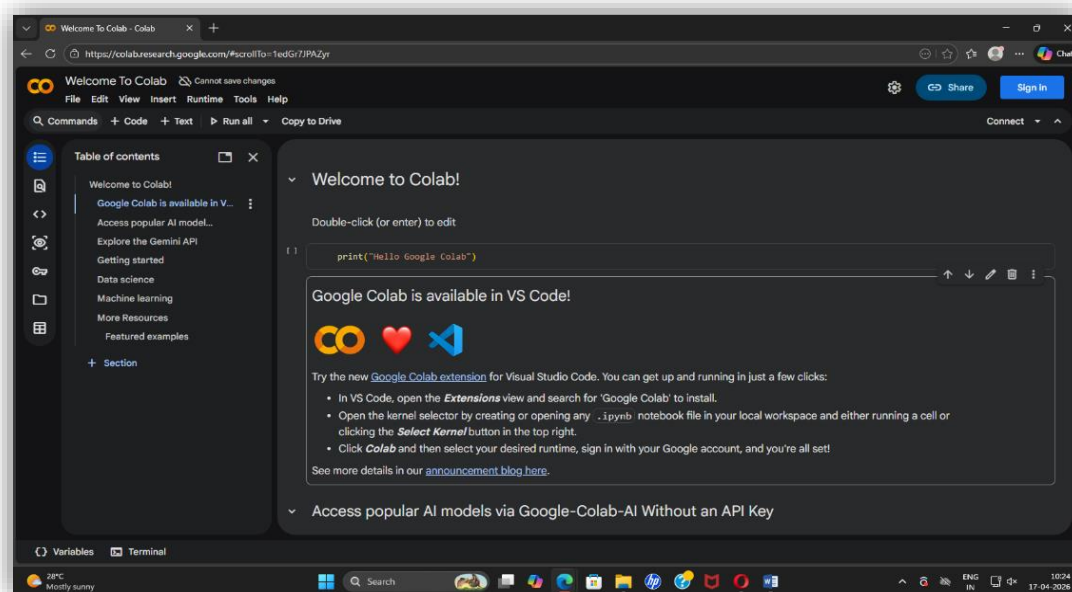
To start Jupyter Notebook:

jupyter notebook--allow-root



D. Google Colaboratory (Colab)

- A cloud-based Python environment provided by Google.
- No installation required.
- Allows users to write and execute Python code using a web browser.
- Useful for collaborative work and access to free computing resources.



DATA SCIENCE

Data is raw information in the form of facts, numbers, symbols, figures or observation.

Data Science is a field that process and analyse data with meaningful output. In short, data is input and data science is process on input to get meaningful output.

A **Dataset** is a complete collection of all observations arranged in some order.

Types of Data:

- **Structured:** Organized in tables (e.g., databases, spreadsheets).
- **Unstructured:** Text, images, videos, social media posts.
- **Qualitative:** Descriptive (e.g. customer reviews).
- **Quantitative:** Numerical (e.g. sales figures).

PYTHON LIBRARIES FOR DATA SCIENCE

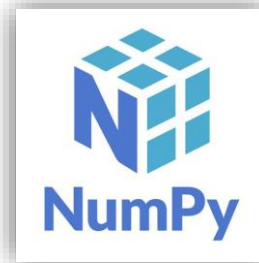
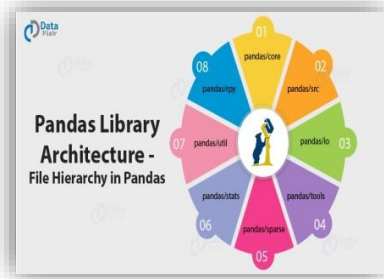
There are several libraries that have been developed for data science and machine learning.

Following are a list of Python libraries:

NumPy, Pandas, Matplotlib, Seaborn, Scikit-learn, TensorFlow, SciPy, jupyter and PyTorch.

Python libraries for Data Processing

- **NumPy** – It stands for Numerical Python. The Foundation for numerical computing, supports large ,multi-dimensional arrays and mathematical functions. Also used in scientific computing.
- **Pandas** – This is most essential for data science which is Powerful for data manipulation, cleaning, filtering and analysis with DataFrames for structured data.
 - ❖ It provides DataFrame and Series objects for tabular data.
 - ❖ They are ideal for cleaning, transforming, and analysing structured datasets.
 - ❖ Which support for CSV, Excel, SQL, JSON, and more.
- **SciPy** : It stands for Scientific Python. It provides advanced modules for integration, linear algebra and statistics.



Python libraries for Data Visualization

The most widely used Python libraries for **Data Visualization** are Matplotlib, Seaborn, Plotly, Bokeh, Altair and ggplot

- **Matplotlib**

- ❖ Matplotlib is the oldest and most widely used plotting library.
- ❖ It standard library used for creating line plots, bar charts, scatter plots, histograms, etc'
- ❖ Recommended highly customizable but verbose.
- ❖ Best for static, publication-quality plots.

- **Seaborn**

- ❖ It built on top of Matplotlib.
- ❖ Seaborn simplifies statistical data visualization (heatmaps, violin plots, pair plots).
- ❖ Very useful for quick, aesthetically pleasing charts.

- **Plotly**

- ❖ It is used for interactive charts and dashboards.
- ❖ It works well in Jupyter notebooks and web apps.
- ❖ Plotly supports 3D plots, maps and animations.

- **Bokeh**

- ❖ Bokehcreating interactive, browser-based visualizations.
- ❖ It integrates with big data tools like Dask.
- ❖ It is ideal for dashboards and streaming data.

- **Altair**

- ❖ Altair declarative syntax based on Vega/Vega-Lite.
- ❖ It is very easy to build complex charts with minimal code.

- ❖ It has capability for statistical visualization and storytelling.

- **ggplot (plotnine in Python)**

- ❖ ggplot inspired by R's ggplot2.
- ❖ It is useful for layered, complex plots and Graphics approach.

Python libraries for Data Extraction

The most widely used Python libraries for **Data Extraction** are BeautifulSoup, Scrapy, Requests, PyPDF2, pdfplumber and Tabula.

- **BeautifulSoup**

- ❖ It is Parsing library in Python used for HTML and XML documents.
- ❖ It's very easy to use with requests for fetching web pages and small-scale web scraping tasks.

- **Scrapy**

- ❖ Scrapy is a full-fledged web scraping framework.
- ❖ It handles crawling, scraping and exporting data.
- ❖ Also supports large-scale projects with built-in pipeline management.

- **Requests**

- ❖ It simplifies HTTP requests (GET, POST, etc.)
- ❖ Also used with BeautifulSoup for fetching and parsing.

- **PyPDF2**

- ❖ It extracts text and metadata from PDF files.
- ❖ PyPDF2 can merge, split and manipulate PDFs.

- **pdfplumber**

- ❖ More accurate text extraction from PDFs.
- ❖ Handles tables and structured layouts better than PyPDF2.

- **Tabula-py**

- ❖ Tabula-py extracts tables from PDFs into Pandas DataFrames.
- ❖ It is useful for financial reports and tabular documents.

Python libraries for Machine Learning

The most important Python libraries for machine learning are Scikit-learn, TensorFlow, PyTorch, XGBoost, LightGBM, and Keras.

- **Scikit-learn**

- ❖ Scikit-learn is best for beginners and classical Machine Learning algorithms (regression, classification, clustering).
- ❖ It provides pre-processing, feature selection and model evaluation tools.
- ❖ It's widely used in academia and industry.

- **NumPy & Pandas**

- ❖ NumPy & Pandas are foundation libraries for numerical computing and data manipulation.
- ❖ They are essential for preparing datasets before Machine Learning model training.

- **TensorFlow**

- ❖ TensorFlow developed by Google, widely used for deep learning and production deployment.
- ❖ It supports neural networks, reinforcement learning and distributed training.
- ❖ Also integrates with **Keras** for easier model building.

- **PyTorch**

- ❖ PyTorch developed by Meta, popular in research and industry.
- ❖ It makes experimentation easier with dynamic computation graphs.
- ❖ It is a strong ecosystem for NLP (Hugging Face Transformers) and computer vision.

- **Keras**

- ❖ Keras is high-level API that simplifies TensorFlow usage.
- ❖ It is very useful for rapid prototyping and beginners.

Introduction to DataFrame

A DataFrame in Python is a two-dimensional, size-mutable, and potentially heterogeneous tabular data structure with labeled axes (rows and columns). It is a fundamental data structure in the pandas library, widely used in data science, machine learning, and data analysis.

It's one of the most commonly used tools for handling data and makes it easy to organize, analyze and manipulate data. It can store different types of data such as numbers, text and dates across its columns. The main parts of a DataFrame are:

- **Data:** Actual values in the table.
- **Rows:** Labels that identify each row.
- **Columns:** Labels that define each data category.

Dictionary

A **dictionary** in Python is a collection of key-value pairs.

- Keys represent column names
- Values represent data lists

In pandas, dictionaries are commonly used to create DataFrames.

Example of Dictionary

```
data= {
'name': ['Ronny', 'Rocky', 'Charlie'],
'age': [25, 30, 28],
'city': ['Nashik', 'Pune', 'Patna']
}
```

- 'name', 'age', 'city' → Keys (columns)
- Lists → Values (data rows)

1. Creating a data frames (Using Dictionary):

```
import pandas as pd
# Creating a DataFrame from a dictionary
data = {'name': ['Ronny', 'Rocky', 'Charlie'],
        'age': [25, 30, 28],
        'city': ['Nashik', 'Pune', 'Patna']}
df = pd.DataFrame(data)
print(df)
```

Output:

name	age	city
------	-----	------

0	Ronny	25	Nashik
1	Rocky	30	Pune
2	Charlie	28	Patna

View Data Frame:

Pandas use the **loc** attribute to return one or more specified row(s)

Example:

```
print(df.loc[0])      #Return Row 0
```

Output:

name	Ronny
age	25
city	Nashik
Name: 0, dtype: object	

Named Indexes

With the **index** argument, you can name your own indexes.

```
import pandas as pd
```

```
# Creating a DataFrame from a dictionary
```

```
data = {'name': ['Alka', 'Bunny', 'Charlie'],
```

```
        'age': [25, 30, 28],
```

```
        'city': ['Nashik', 'Pune', 'Patna']}
```

```
df = pd.DataFrame(data, index = ["Value1", "Value2", "Value3"])
```

```
print(df)
```

Output:

	name	age	city
Value1	Alka	25	Nashik
Value2	Bunny	30	Pune
Value3	Charlie	28	Patna

Locate Named Indexes

Use the named index in the **loc** attribute to return the specified row(s).

```
print(df.loc["Value2"])#refer to the named index
```

Output:

name	Bunny
age	30
city	Pune
Name: Value2, dtype: object	

Creating and view data frames (Using .csv file):

If your data sets are stored in a file, Pandas can load them into a DataFrame.

A **CSV (Comma-Separated Values) file** is commonly used to store tabular data.

Step 1: Create a CSV File

First, create a file named **data.csv** with sample data like this:

```
Name,Age,City
Asha,20,Mumbai
Ravi,22,Delhi
```

Step 2: Load CSV File into DataFrame

Import pandas as pd

```
df=pd.read_csv('data.csv')
```

```
print(df)
```

Output

```
   Name  Age  City
0  Asha   20  Mumbai
1  Ravi   22  Delhi
```

Viewing Large DataFrames

If the DataFrame contains many rows, Pandas will not display all of them by default.

It usually shows only the first few and last few rows.

To display the entire DataFrame, use:

```
print(df.to_string())
```

Activity 1: Create empty dataframe and add records

```
#Import the library
```

```
import pandas as pd
```

```
#Create an empty data frame with column names
```

```
df = pd.DataFrame(columns = ['company','model','year'])
```

```
#Add records
```

```
df.loc[0] = ['Tata', 'Nexon', 2017]
```

```
df.loc[1] = ['MG', 'Astor', 2021]
```

```
df.loc[2] = ['KIA', 'Seltos', 2019]
```

```
df.loc[3] = ['Hyundai', 'Creta', 2015]
```

```
#Print the dataframe
```

```
df
```

Activity 2: Create dataframe using numpy array

```
#Import the library
```

```
import numpy as np
```

```
# Pass a 2D numpy array - each row is the corresponding row in the dataframe
```

```
data = np.array([['Tata','Nexon',2017],  
  
['MG','Astor',2021],  
  
['KIA','Seltos', 2019],  
  
['Hyundai','Creta',2015]])  
  
# pass column names in the columns parameter of the constructor  
  
df = pd.DataFrame(data, columns = ['company', 'model','year'])  
  
df
```

Activity 3: Create dataframe using list

Each element of the list is one record in the dataframe. The column headers are passed separately.

```
# Import pandas library  
  
import pandas as pd  
  
# Create a list of lists  
  
data = [['Tata','Nexon',2017],['MG','Astor',2021],['KIA','Seltos', 2019],['Hyundai','C  
  
reta',2015]]  
  
# Create the pandas DataFrame  
  
df = pd.DataFrame(data, columns = ['company', 'model','year'])  
  
# print the dataframe.  
  
df
```

Activity 4: Create dataframe using dictionary with lists

```
#Import the library
```

```
import pandas as pd

#Create the dictionary

data = {'company': ['Tata','MG','KIA','Hyundai'],

'model':['Nexon','Astor','Seltos','Creta'],

'year': [2017,2021,2019,2015]

}

#Create the dataframe

df = pd.DataFrame.from_dict(data)

#Print the dataframe

df
```

Activity 5: Create dataframe using list of dictionary

```
import pandas as pd

#Each dictionary is a record in the dataframe.

#Dictionary Keys become Column names in the dataframe. Dictionary values become the

values of columns

data = [{'company':'Tata','model':'Nexon','year':2017},

{'company':'MG','model':'Astor','year':2021},

{'company':'KIA','model':'Seltos','year':2019},

{'company':'Hyundai','model':'Creta','year':2015}]

df = pd.DataFrame(data)
```

df

Activity 6: Create dataframe using zip() function

```
import pandas as pd
```

```
#Company list
```

```
company = ['Tata','MG','KIA','Hyundai']
```

```
#Model list
```

```
model = ['Nexon','Astor','Seltos','Creta']
```

```
#Year list
```

```
year = [2017,2021,2019,2015]
```

```
# and merge them by using zip().
```

```
data = list(zip(company, model,year))
```

```
# Create pandas Dataframe using data
```

```
df = pd.DataFrame(data,columns = ['company','model','year'])
```

```
# Print dataframe
```

df

Activity 7: Understand the data

Apply functions info(), describe(), shape, size, loc, sort_values, value_counts() on the dataframe object.

Activity 8: Adding rows and columns with Invalid, duplicate or missing data

```
#Add rows to the dataset with empty or invalid values.
```

```
df.loc[4] = ['Honda', 'Jazz', None]
```

```
df.loc[5]= [None, None, None]
```

```
df.loc[6]=['Toyota', None , 2018]
```

```
df.loc[7]=[ 'Tata','Nexon',2017]
```

```
#Insert empty column
```

```
df["newcolumn"] = None
```

```
df
```

Note: a new column can be added with specific values. For example, if we want the new column to have

model name + year name, the command will be:

```
df['new']=df['model']+' '+df['year'].astype(str) #Convert year to string
```

Activity 9: Check for NULL and duplicate values

```
#View null values
```

```
df.isnull() # df.notnull() shows all values that are not null
```

```
df.duplicated() #Shows the rows with duplicate entries
```

Activity 10: Replace NULL values with fillna() function

To replace all Empty values with the string “No Value Available”

```
#DataFrame.fillna() to replace Null values in dataframe
```

```
df.fillna("No Value Available", inplace = True)
```

```
df
```

Activity 11: Drop a column from the dataframe

To drop the column named “newcolumn”

```
df.drop(columns='newcolumn', axis=1, inplace=True)
```

```
df
```

Activity 12: Load the iris dataset

If you are using a local machine to run python code, download the iris dataset and type the following code. The pathname is the location of the dataset in the local drive.

```
import pandas as pd
```

```
df=pd.read_csv("\iris.csv") #Give the path as per the location in your machine
```

```
df
```

If you are using google colab, then run the following code to load the dataset.

```
from google.colab import files
```

```
iris_uploaded = files.upload()
```

This will prompt you for selecting the csv file to be uploaded. To load the file, run the following code:

```
import io
```

```
df = pd.read_csv(io.BytesIO(iris_uploaded['Iris.csv']))
```

```
df
```

Activity 13: Understand the iris dataset

Type and execute the following commands:

i. df.shape

Answer the following:

a. How many rows and columns in the dataset?

ii. `df.describe()`

Answer the following:

a. Mean values

b. Standard Deviation

c. Minimum Values

d. Maximum Values

iii. `df.info()`

Answer the following:

a. Does any column have null entries?

b. How many columns have numeric data?

c. How many columns have categorical data?

iv. `df.dtypes`

v. `df.head()`, `df.head(20)`

vi. `df.tail()`

vii. `df.sample(10)`

viii. `df.size`

ix. `df.columns`

What are the names of the columns?

x. For each column name, run the following command:

```
df['columnname'].value_counts()
```

xi. View rows 10 to 20

```
sliced_data=df[10:20]
```

```
print(sliced_data)
```

xii. Selecting specific columns

```
specific_data=df[["Id","Species"]]
```

```
print(specific_data)
```

SET A

1. Write a Python program to check whether a given number is a palindrome or not.
2. Write a Python program to check whether a given number is a prime number or not.
3. Write a Python program to find the factorial of a given number.
4. Write a Python program to generate the Fibonacci series up to n terms.
5. Write a Python program to check whether a given number is an Armstrong number or not.

SET B

1. Create a dictionary that stores student names as values for the key '**Name**', marks as values for the key '**Marks**', and city names as values for the key '**City**'. Convert the dictionary into a DataFrame and display it.
2. Write a Python program to create a DataFrame and display the first row and second row using the loc attribute.
3. Write a Python program to read a CSV file named data.csv and display its contents as a DataFrame.

SET C

1. Write a Python program to find the shape, size, datatypes of the dataframe object.
2. Write a Python program to view basic statistical details of the data.
3. Write a Python program to get the number of observations, missing values.

4. Write a Python program to view basic statistical details of the data.
5. Write a Python program to Add 5 rows with duplicate values and missing values. Add a column 'remarks' with empty values. Display the data

Assignment Evaluation

0: Not Done		1: Incomplete		2: Late Complete	
3: Needs improvement		4: Complete		5: Well Done	

Signature of the Instructor

Date

Assignment 2: Statistical Data Analysis

Objectives

- To understand how to apply statistical methods to analyze data.
- To use Python libraries like NumPy, Pandas, SciPy, and Matplotlib to analyze real data.
- To write simple Python programs that use functions from these statistical libraries.
- To understand importance of data analysis using statistical methods in various fields.

Reading References:

Statistical Data Analysis

Statistical Data Analysis in data science involves collecting, organizing, analysing, and interpreting data to discover patterns, trends, and relationships, enabling data-driven decisions. It acts as a bridge between raw data and actionable insights to draw conclusions and make predictions about larger populations.

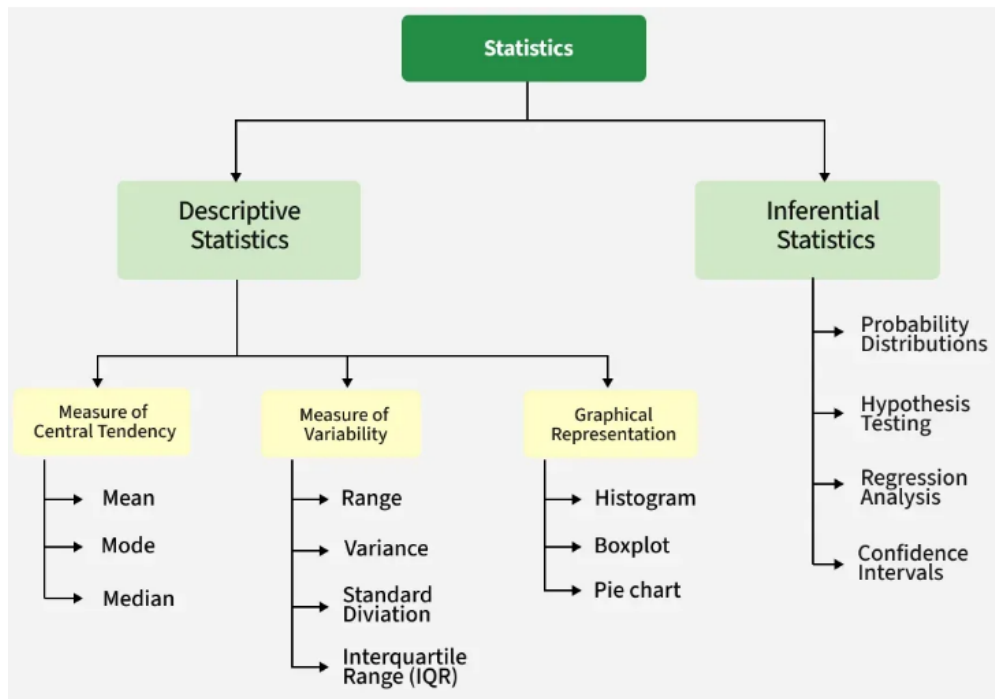
Choosing Python Statistics Libraries

Python has a rich ecosystem of libraries for statistics, data analysis, and machine learning. Below are the most important ones and their roles.

Library	Purpose	Description
NumPy	Numerical computing	Provides fast N-dimensional arrays, mathematical functions, and random number generation; foundation of scientific computing in Python.

SciPy	Scientific computing & statistics	Builds on NumPy to offer statistical distributions, hypothesis tests, optimization, and advanced scientific functions.
pandas	Data analysis	Offers DataFrame structures for data manipulation, cleaning, and descriptive statistical analysis.
statsmodels	Statistical modeling	Provides regression models, ANOVA, time series analysis, and classical statistical inference tools.
scikit-learn	Machine learning	Supplies tools for classification, regression, clustering, preprocessing, and model evaluation.
Pingouin	Statistical testing	Simple and user-friendly library for common statistical tests, effect sizes, and reliability analysis.
PyMC	Bayesian statistics	Enables probabilistic modeling and Bayesian inference using MCMC and advanced sampling methods.

Types of Statistics



Descriptive Statistics

Descriptive statistics are methods used to **summarize, organize, and describe data** in a meaningful way. It finds key characteristics, such as central tendency (mean, median, mode), variability (range, standard deviation), and distribution shape. They provide a concise, actionable snapshot of raw data through numerical calculations or visual aids like graphs and tables.

Types of Measures

1. Central tendency tells you about the centers of the data. Useful measures include the mean, median, and mode.

a. The **mean** is the central tendency measure defined as the sum of all observations divided by the total number of observations.

For a dataset $X = \{x_1, x_2, \dots, x_n\}$:

$$\mu = \frac{1}{n} \sum_{i=1}^n x_i$$

Where:

n = number of observations

x_i = each observation

Example-Python Code

```
import numpy as np
data = [10, 20, 30, 40, 50]
mean_value = np.mean(data)
print("Mean:", mean_value)
```

- b. The **median** is the value that divides an ordered dataset into two equal halves.

For ordered dataset:

If n is odd:

$$\text{Median} = x_{\frac{n+1}{2}}$$

If n is even:

$$\text{Median} = \frac{x_{\frac{n}{2}} + x_{\frac{n}{2}+1}}{2}$$

Example-Python Code

```
import numpy as np
data = [10, 20, 30, 40, 50]
median_value = np.median(data)
print("Median:", median_value)
```

- c. The **mode** is the value that occurs with the highest frequency in a dataset.

Mathematically:

$$\text{Mode} = x_k \text{ such that } f(x_k) = \max(f(x_i))$$

Where $f(x_i)$ represents the frequency of each value.

Example-Python Code

```
import statistics
data = [10, 20, 20, 30, 40]
mode_value = statistics.mode(data)
print("Mode:", mode_value)
```

2. **Variability** tells you about the spread of the data. Useful measures include variance and standard deviation, range and interquartile range.

- a. Variance measures the **dispersion of data points** around the mean. It is the expected

value of the squared deviation from the mean.

For a dataset $X = \{x_1, x_2, \dots, x_n\}$:

Population Variance

$$\sigma^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \mu)^2$$

Sample Variance

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$$

Where:

μ = population mean

$\bar{x}$ = sample mean

n = number of observations

Example-Python Code

```
import numpy as np
data = [10, 20, 30, 40, 50]
# Population variance
pop_variance = np.var(data)
# Sample variance
sample_variance = np.var(data, ddof=1)
print("Population Variance:", pop_variance)
print("Sample Variance:", sample_variance)
```

- b. Standard deviation is the **square root of variance**.

$$\sigma = \sqrt{\sigma^2}$$

$$s = \sqrt{s^2}$$

Example-Python Code

```
import numpy as np
data = [10, 20, 30, 40, 50]
std_population = np.std(data)
```



```
std_sample = np.std(data, ddof=1)
print("Population Std Dev:", std_population)
print("Sample Std Dev:", std_sample)
```

- c. Range is the simplest dispersion measure defined as:

$$\text{Range} = X_{\max} - X_{\min}$$

Example-Python Code

```
data = [10, 20, 30, 40, 50]
range_value = max(data) - min(data)
print("Range:", range_value)
```

- d. The **Interquartile Range (IQR)** measures the spread of the middle 50% of the data.

$$IQR = Q_3 - Q_1$$

Where:

Q_1 = First quartile (25th percentile)

Q_3 = Third quartile (75th percentile)

Example-Python Code

```
import numpy as np
data = [10, 20, 30, 40, 50]
Q1 = np.percentile(data, 25)
Q3 = np.percentile(data, 75)
IQR = Q3 - Q1
print("Q1:", Q1)
print("Q3:", Q3)
print("IQR:", IQR)
```

3. **Correlation or joint variability** tells you about the relation between a pair of variables in a dataset. Useful measures include covariance and the correlation coefficient.

Covariance Example

```
import numpy as np
X = [10, 20, 30, 40, 50]
Y = [15, 25, 35, 45, 55]
# Sample covariance matrix
cov_matrix = np.cov(X, Y)
print("Covariance Matrix:\n", cov_matrix)
print("Sample Covariance:", cov_matrix[0,1])
```

Correlation Coefficient (Pearson's r) Example

```
import numpy as np

X = [10, 20, 30, 40, 50]
Y = [15, 25, 35, 45, 55]

# Pearson correlation coefficient
correlation = np.corrcoef(X, Y)
print("Correlation Matrix:\n", correlation)
print("Pearson r:", correlation[0,1])
```

Proximity Measures

Proximity measures are basically mathematical techniques that are used to check *similarity* between objects or one can say *dissimilarity* between objects. These measures are based on distance measures, although it can be based on probabilities also.

Distance Measures

Distance measures are mathematical functions that quantify the similarity or dissimilarity between data points, acting as a foundational component in statistics and machine learning for clustering, classification, and anomaly detection. By calculating how close data points are in feature space, these metrics, such as **Euclidean or Manhattan distance** enable algorithms to group similar items and distinguish different ones.

In simple way:

- The smaller the distance → the more similar the objects
- The larger the distance → the more different they are

a. Euclidean distance: Euclidean Distance is a metric used to measure the straight-line distance between two points in an *n-dimensional Euclidean space*. It is derived from the Euclid geometry and is based on the Pythagorean theorem.

Mathematical Definition

For two points:

$$P(x_1, x_2, \dots, x_n)$$

$$Q(y_1, y_2, \dots, y_n)$$

The Euclidean distance is defined as:

$$d(P, Q) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

In 2-Dimensional Space

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

In 3-Dimensional Space

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2 + (z_2 - z_1)^2}$$

Example-Python Code

```
import numpy as np
# Define two vectors
x = np.array([1, 2, 3])
y = np.array([4, 5, 6])
# Euclidean distance
distance = np.linalg.norm(x - y)
print("Euclidean Distance:", distance)
```

b. Manhattan distance is a metric that measures the distance between two points as the sum of the absolute differences of their coordinates.

For two vectors:

$$\mathbf{x} = (x_1, x_2, \dots, x_n)$$

$$\mathbf{y} = (y_1, y_2, \dots, y_n)$$

The Manhattan distance is defined as:

$$d(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^n |x_i - y_i|$$

Example-Python Code

```
import numpy as np
x = np.array([1, 2, 3])
y = np.array([4, 5, 6])
```

```
distance = np.sum(np.abs(x - y))  
print("Manhattan Distance:", distance)
```

Core Roles and Applications of Distance Measures

- **Clustering Algorithms:** K-Means, hierarchical clustering, and DBSCAN, where smaller distances indicate higher similarity.
- **Classification (Supervised Learning):** Algorithms like K-Nearest Neighbors (K-NN) use distances to classify data points based on their closest, labelled neighbours.
- **Dimensionality Reduction:** Techniques like t-SNE, UMAP, and PCA rely on measuring distances
- **Anomaly/Outlier Detection:** Points with unusually large distances from other data points are identified as outliers or anomalies.
- **Natural Language Processing (NLP) & Recommendation Systems:** Measures like Cosine similarity determine how similar documents or user preferences are.

Outliers

An outlier is an observation that significantly deviates from other observations in a dataset, potentially indicating variability in measurement, experimental error, or rare events.

Common Statistical Methods for Detecting Outliers

1. Z-Score Method
2. Interquartile Range (IQR) Method
3. Boxplot Method
4. Percentile Method

Types of Outliers

1. Point Outlier – Single extreme value
2. Contextual Outlier – Abnormal in a specific context
3. Collective Outlier – Group of abnormal observations

Example-Python Code (outlier detection using z_score)

```
import numpy as np  
data = np.array([10, 12, 14, 15, 18, 100]) # 100 is an outlier
```

```

mean = np.mean(data)
std = np.std(data)
z_scores = (data - mean) / std
outliers = data[np.abs(z_scores) > 3]
print("Z-scores:", z_scores)
print("Outliers:", outliers)

```

Population and Samples

A **population** is the complete set of all possible observations or measurements of interest in a statistical study.

Mathematically, a population can be represented as:

$$\mathcal{P} = \{x_1, x_2, \dots, x_N\}$$

Where:

N = total number of elements in the population

Population parameters include:

- Mean (μ)
- Variance (σ^2)
- Standard deviation (σ)

A **sample** is a subset of the population selected to estimate population parameters.

$$\mathcal{S} = \{x_1, x_2, \dots, x_n\}, n < N$$

Where:

n = sample size

Sample statistics include:

- Sample mean ($\bar{x}$)
- Sample variance (s^2)
- Sample standard deviation (s)

Example: Population and Random Sample

```

import numpy as np
# Create a population
population = np.arange(1, 101) # Population: 1 to 100
N = len(population)
# Population parameters
pop_mean = np.mean(population)
pop_var = np.var(population)

```

```
print("Population Size:", N)
print("Population Mean:", pop_mean)
print("Population Variance:", pop_var)
```

Self Activities

Example : Use of describe() function

```
import pandas as pd
data = {
    "Employee": [f"E{i}" for i in range(1, 16)],
    "Age": [22, 25, 29, 30, 31, 35, 38, 40, 42, 45, 47, 50, 52, 55, 60]
}
df = pd.DataFrame(data)
print("Employee DataFrame:\n")
print(df)
print("\n--- Descriptive Statistics for Age ---")
stats = df["Age"].describe()
print(stats)
```

Output

Employee DataFrame:

	Employee	Age
0	E1	22
1	E2	25
2	E3	29
3	E4	30
4	E5	31
5	E6	35
6	E7	38
7	E8	40
8	E9	42
9	E10	45
10	E11	47
11	E12	50

```
12    E13    52
13    E14    55
14    E15    60
```

Descriptive Statistics for Age

```
count    15.000000
mean     39.867
std      11.806
min      22.000
25%      30.000
50%      40.000
75%      50.000
max      60.000
```

Name: Age, dtype: float64

Example: Calculate mean, median,mode

```
import pandas as pd

data = {
    "Math": [85, 90, 78, 92, 88, 76, 95, 89],
    "Science": [88, 79, 85, 91, 87, 80, 93, 84],
    "English": [75, 85, 82, 78, 90, 88, 84, 79]
}

# Create DataFrame
df = pd.DataFrame(data)

# Display DataFrame
print("Student Marks DataFrame:\n")
print(df)

print("\n Column-wise Statistics")

# Mean of each column
print("\nMean:")
```

```

print(df.mean())

# Median of each column
print("\nMedian:")
print(df.median())

# Mode of each column
print("\nMode:")
print(df.mode())

print("\n Overall Statistics")

# Example for one subject
math_mean = df["Math"].mean()
math_median = df["Math"].median()
math_mode = df["Math"].mode()[0]

print("\nMath Mean:", math_mean)
print("Math Median:", math_median)
print("Math Mode:", math_mode)

```

Output

Student Marks DataFrame:

	Math	Science	English
0	85	88	75
1	90	79	85
2	78	85	82
3	92	91	78
4	88	87	90
5	76	80	88
6	95	93	84
7	89	84	79

Column-wise Statistics

Mean:

Math 86.625

Science 85.875

English 82.625

dtype: float64

Median:

Math 88.5

Science 86.0

English 83.0

dtype: float64

Mode:

	Math	Science	English
0	76	79	75
1	78	80	78
2	85	84	79
3	88	85	82
4	89	87	84
5	90	88	85
6	92	91	88
7	95	93	90

Overall Statistics

Math Mean: 86.625

Math Median: 88.5

Math Mode: 76

Example :Calculate Range Variance and Standard Deviation

```
import pandas as pd

# Monthly sales data (in thousands)
data = {
    "Month": ["Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"],
    "Sales": [120, 135, 150, 160, 145, 155, 165, 170, 160, 175, 180, 190]
}

# Create DataFrame
df = pd.DataFrame(data)

# Display DataFrame
print("Monthly Sales Data:\n")
print(df)

# Calculate Range
sales_range = df["Sales"].max() - df["Sales"].min()

# Calculate Variance
sales_variance = df["Sales"].var()

# Calculate Standard Deviation
sales_std = df["Sales"].std()

# Display results
print("\n Sales Statistics")
print("Range:", sales_range)
print("Variance:", round(sales_variance, 2))
print("Standard Deviation:", round(sales_std, 2))
```

Output

Monthly Sales Data:

	Month	Sales
0	Jan	120
1	Feb	135
2	Mar	150
3	Apr	160
4	May	145
5	Jun	155
6	Jul	165
7	Aug	170
8	Sep	160
9	Oct	175
10	Nov	180
11	Dec	190

Sales Statistics

Range: 70

Variance: 480.3

Standard Deviation: 21.92

Example: Calculate quartiles, percentiles and detect outliers using IQR and Z-Score.

```
import pandas as pd
```

```
import numpy as np
```

```
from scipy import stats
```

```
# Sample dataset: Monthly sales (in thousands)
```

```
data = {
```

```
    "Month": ["Jan", "Feb", "Mar", "Apr", "May", "Jun",
```

```
             "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"],
```

```
    "Sales": [120, 135, 150, 160, 145, 155, 165, 170, 160, 175, 180, 300]
```

```
}
```

```

# Create DataFrame
df = pd.DataFrame(data)

print("Monthly Sales Data:\n")
print(df)

# Quartiles
Q1 = df["Sales"].quantile(0.25)
Q2 = df["Sales"].quantile(0.50) # Median
Q3 = df["Sales"].quantile(0.75)

print("\n Quartiles ")
print(f"Q1 (25th percentile): {Q1}")
print(f"Q2 (50th percentile / Median): {Q2}")
print(f"Q3 (75th percentile): {Q3}")

# Selected Percentiles
percentiles = [25, 50, 75, 90]
perc_values = np.percentile(df["Sales"], percentiles)
print("\n Selected Percentiles ")
for p, val in zip(percentiles, perc_values):
    print(f"{p}th percentile: {val}")

# Outlier Detection using IQR
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
outliers_iqr = df[(df["Sales"] < lower_bound) | (df["Sales"] > upper_bound)]

print("\n Outliers using IQR method ")
print(outliers_iqr)

# Outlier Detection using Z-score
z_scores = np.abs(stats.zscore(df["Sales"]))

```

```
threshold = 3 # Common threshold for Z-score method
```

```
outliers_z = df[z_scores > threshold]
```

```
print("\n Outliers using Z-score method ")
```

```
print(outliers_z)
```

Output

Monthly Sales Data:

Month Sales

0 Jan 120

1 Feb 135

2 Mar 150

3 Apr 160

4 May 145

5 Jun 155

6 Jul 165

7 Aug 170

8 Sep 160

9 Oct 175

10 Nov 180

11 Dec 300

Quartiles

Q1 (25th percentile): 145.0

Q2 (50th percentile / Median): 160.0

Q3 (75th percentile): 175.0

Selected Percentiles

25th percentile: 145.0

50th percentile: 160.0

75th percentile: 175.0

90th percentile: 210.0

Outliers using IQR method

Month Sales

11 Dec 300

Outliers using Z-score method

Month Sales

11 Dec 300

Example :Calculate Euclidean distance between 2 data points.

```
import numpy as np
# Define two points as arrays
point1 = np.array([3, 4])
point2 = np.array([7, 1])
# Compute Euclidean distance
distance = np.linalg.norm(point2 - point1)
# Display the result
print(f"The Euclidean distance between {point1} and {point2} is:
{distance:.4f}")
```

Output

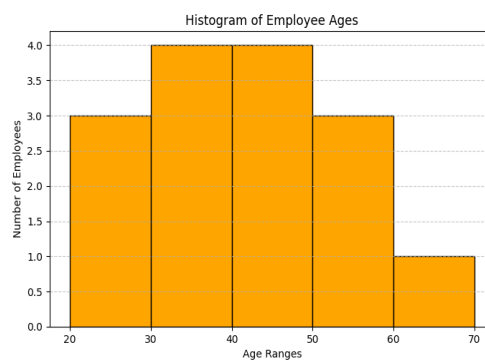
The Euclidean distance between [3 4] and [7 1] is: 5.0000

Example :Compute and display Histogram using custom bins.

```
import numpy as np
import matplotlib.pyplot as plt
ages = np.array([22, 25, 29, 30, 31, 35, 38, 40, 42, 45, 47, 50, 52, 55, 60])
# Define custom bins
bins = [20, 30, 40, 50, 60, 70]
hist, bin_edges = np.histogram(ages, bins=bins)
print("Age Histogram (Frequency in each bin):")
for i in range(len(hist)):
    print(f"Age {bin_edges[i]} - {bin_edges[i+1]}: {hist[i]}")
```

```
plt.figure(figsize=(8,5))
plt.hist(ages, bins=bins, edgecolor='black', color='orange')
plt.title("Histogram of Employee Ages")
plt.xlabel("Age Ranges")
plt.ylabel("Number of Employees")
plt.xticks(bins) # Show bin edges on x-axis
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```

Output

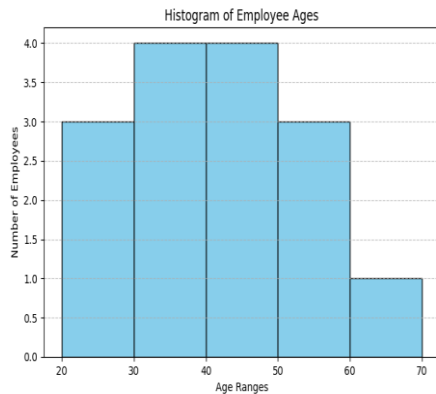


Lab Assignments

SET A:

1. Create a student DataFrame which contains student_id, name and marks. Write a Python program to calculate mean, median, and mode of given data.
2. Create a DataFrame containing the monthly sales (in thousands) of a company for 12 months. Write a Python program to calculate range, variance, and standard deviation of sales data.
3. Create a DataFrame containing the salaries (in thousands) of 25 employees in a company. Write a Python program to calculate the quartiles (Q1, Q2, Q3) and selected percentiles (25th, 50th, 75th, 90th), and detect outliers using both the IQR and Z-score methods.
4. Create a DataFrame containing 2D coordinates of multiple data points. Write a Python program to select any two points and compute the Euclidean distance between them using `numpy.linalg.norm()`, showing both the difference vector and the final distance.

5. Create a NumPy array of the ages of 15 employees and write a Python program to compute and display the histogram using custom bins, showing the frequency of ages in each bin, and plot the histogram using matplotlib with labelled axes.



SET B:

1. Write a Python program to compute **complete descriptive statistics** for the numeric columns (Salary) using **pandas**, including mean, median, variance, standard deviation, minimum, maximum, quartiles, and count.[Use employee.csv from <https://github.com>]
2. Download any dataset from UCI. Read this csv file using read_csv() function. Describe the dataset using appropriate function. Display mean value of numeric attributes. Check any data values are missing or not.
3. Create an Employee DataFrame with columns Employee_ID, Department, Age, and Salary. Write a Python program to implement
 - Simple Random Sampling (select 5 employees randomly),
 - Systematic Sampling (select every 3rd employee), and
 - Stratified Sampling : Group by Department and randomly select 1 employee from each department
4. Download iris dataset file. Read this csv file using read_csv() function. Take samples from entire dataset. Display maximum and minimum values of all numeric attributes, find number of records for each distinct value of class attribute.

SET C:

1. Create a Python program to compute Euclidean, Manhattan distance and generate a similarity/dissimilarity matrix for multidimensional data.
2. Write a Python program that takes a dataset of experiments and computes **weighted arithmetic, harmonic, and geometric means** along any chosen axis. Use vectorized NumPy operations and allows custom weights.
3. Write a python program to compute the **normalized cross-correlation** between two multi-dimensional arrays along a specified axis and report the **lag of maximum correlation**.
4. Write a Python program to identify outliers in a dataset using IQR, Z-score, and Modified Z-score and analyse how removing them affects the mean, median, and standard deviation.

Assignment Evaluation

0: Not Done		1: Incomplete		2: Late Complete	
3: Needs improvement		4: Complete		5: Well Done	

Signature of the Instructor

Date

Assignment 3: Data Preprocessing

Objectives

- To understand the **importance of preparing raw data** before analysis.
- To study the **process of data wrangling and cleaning**.
- To implement **Python-based preprocessing techniques** on different datasets.

Prerequisite Knowledge

Students are expected to study the following concepts prior to performing this experiment:

- Importing required **Python libraries and datasets**
- Working with **pandas DataFrame structure**
- Fundamentals of **data preprocessing in data science**
- Techniques for **treating missing or inconsistent values**
- Data wrangling/munging operations including
 - Data Cleaning
 - Data Transformation
 - Data Reduction
 - Data Discretization

Reading References:

Introduction: Data Munging / Data Wrangling Operations

Data munging (also called **data wrangling**) means the process of **preparing raw data so that it can be properly used for analysis or machine learning**.

In real life, the data that we collect is usually:

- incomplete
- unorganized
- contains errors
- stored in different formats

Because of this, we cannot directly use raw data for **analysis, visualization, or prediction**.

Therefore, we must **clean, transform, and organize** the data before using it.

This entire preparation process is known as **data munging**.

After munging, the data becomes:

- structured
- meaningful
- easy to analyze
- suitable for machine learning models

Main Operations in Data Munging

The data munging process mainly includes the following steps:

1. Data Cleaning

Removing or correcting:

- missing values
- wrong entries
- duplicate records
- inconsistent formats

This step improves the **quality and accuracy** of the dataset.

2. Data Transformation

Changing data into a **useful mathematical or numerical format**, such as:

- scaling numbers between 0 and 1
- converting text labels into numbers
- normalizing values

This makes the data **suitable for machine learning algorithms**.

3. Data Reduction

Reducing the **size or number of features** in the dataset without losing important information.

This helps in:

- faster computation
- lower memory usage
- simpler models

4. Data Discretization

Converting **continuous numerical values** into **categories or groups**.

Example:

Marks → Poor, Average, Good, Excellent

This makes the data **easier to understand and analyze**.

Steps for Data Preprocessing

To prepare data properly, we follow these main steps:

1. Importing the required libraries

2. Importing the dataset into Python
3. Handling missing or incorrect values
4. Applying data transformation techniques

These steps together form the **complete preprocessing workflow**.

Step 1: Importing the Libraries

Before working with any dataset, we must import the **Python libraries** that provide useful functions.

1. NumPy

- NumPy is used for **numerical calculations**.
- It provides **arrays and mathematical operations**.
- Since most machine learning models work on **arrays**, NumPy is very important.

Example Program Using NumPy

```
import numpy as np
```

```
# Create a NumPy array
```

```
arr = np.array([10, 20, 30, 40])
```

```
print("NumPy Array:", arr)
```

```
print("Sum of elements:", np.sum(arr))
```

```
print("Mean value:", np.mean(arr))
```

Explanation:

- `np.array()` → creates a **NumPy array**
- `np.sum()` → calculates **total of all elements**
- `np.mean()` → calculates **average value**

Output Explanation

NumPy Array: [10 20 30 40]

Sum of elements: 100

Mean value: 25.0

This shows that **NumPy quickly performs mathematical calculations on arrays**.

2. Matplotlib

- Matplotlib is used to **draw graphs and charts**.
- It helps us **visualize the results** of data analysis.
- Examples: line graph, bar chart, histogram, etc.

Example Program Using Matplotlib

```
import matplotlib.pyplot as plt
```

```
# Data for plotting
```

```
marks = [40, 55, 70, 85]
```

```
students = ["A", "B", "C", "D"]
```

```
# Create bar chart
```

```
plt.bar(students, marks)
```

```
plt.title("Student Marks")
```

```
plt.xlabel("Students")
```

```
plt.ylabel("Marks")
```

```
plt.show()
```

Explanation:

- `plt.bar()` → creates a **bar chart**
- `plt.title()` → sets **graph title**
- `plt.xlabel()` and `plt.ylabel()` → label the axes
- `plt.show()` → displays the graph

Output Explanation

The output will be a **bar graph** where:

- X-axis shows **student names**
- Y-axis shows **marks**
- Each bar height represents **marks scored by a student**

This helps us **visually compare performance** instead of reading numbers in a table.

3. Pandas

- Pandas is used to **load and manage datasets**.
- It stores data in a **DataFrame**, which looks like a table with rows and columns.
- It also helps in **cleaning, filtering, and analyzing data**.

Example Program Using Pandas

```
import pandas as pd

# Create a simple dataset
data = {
    "Name": ["Rahul", "Sneha", "Amit"],
    "Marks": [65, 78, 82]
}

df = pd.DataFrame(data)

print("DataFrame:")
print(df)

print("\nAverage Marks:", df["Marks"].mean())
```

Explanation:

- `pd.DataFrame()` → creates a **table-like dataset**
- `df["Marks"].mean()` → calculates **average marks**

Output Explanation

DataFrame:

Name Marks

0 Rahul 65

1 Sneha 78

2 Amit 82

Average Marks: 75.0

This shows:

- Data stored in **rows and columns**

- Pandas can easily **perform calculations on columns**

Syntax for Importing Libraries

```
import numpy as np      # for numerical operations
import pandas as pd     # for dataset handling
from sklearn.impute import SimpleImputer # for missing values
from sklearn.preprocessing import LabelEncoder, OneHotEncoder # for encoding
```

Step 2: Importing the Dataset

After importing libraries, the next step is to **load the dataset into the Python program**. We usually store datasets in **CSV (Comma Separated Values) files**. Pandas provides the function **read_csv()** to read such files.

Basic Syntax to Import Dataset

```
import pandas as pd

data = pd.read_csv("Data_1.csv")
print(data)
```

Explanation:

When the `read_csv()` function executes:

- The CSV file is **loaded into memory as a DataFrame object**
- The data is stored as a **Pandas DataFrame**
- The dataset becomes **ready for cleaning, analysis, and preprocessing**

Later, this DataFrame can also be **converted into NumPy arrays** when required for **machine learning algorithms**.

Understanding DataFrame

A **DataFrame** is:

- a **two-dimensional data structure**
- similar to a **table or spreadsheet**
- contains **rows and columns**
- can store **numbers, text, or mixed data types**

Example: Creating a Simple DataFrame

```
import pandas as pd

data = {
    "calories": [420, 380, 390],
    "duration": [50, 40, 45]
}

df = pd.DataFrame(data)
print(df)
```

Explanation of the Code

- A **dictionary** is created containing two columns:
 - calories
 - duration
- `pd.DataFrame()` converts this dictionary into a **tabular dataset**.
- `print(df)` displays the table.

Output Explanation

```
calories  duration
0      420      50
1      380      40
2      390      45
```

This output shows:

- **3 rows** of data
- **2 columns**
- Each row represents **one observation**

Thus, a DataFrame behaves like a **small spreadsheet inside Python**.

Locating Rows in a DataFrame Using .loc[]

The `.loc[]` method in Pandas is used to **select data using labels**, which means:

- we select rows or columns using their **names**
- we can apply **logical conditions**
- when using a range, the **last value is included**

This method is very useful when we want to **filter specific records** from a dataset.

Example: Selecting Data Using a Condition

Suppose we have a dataset of vehicles that contains:

- Brand name
- Mileage value

We want to **display only those cars** that:

- belong to brand "**Maruti**"
- have **mileage greater than 25**

Example:

```
import pandas as pd
```

```
data = pd.DataFrame({  
    "Brand": ["Maruti", "Hyundai", "Maruti", "Tata"],  
    "Mileage": [28, 18, 30, 22]  
})
```

```
result = data.loc[(data["Brand"] == "Maruti") & (data["Mileage"] > 25)]  
print(result)
```

Explanation of the Code

- `data["Brand"] == "Maruti"` → checks rows where brand is Maruti
- `data["Mileage"] > 25` → checks rows where mileage is greater than 25
- `&` → combines both conditions
- `.loc[]` → returns only the **matching rows**

Output Explanation

	Brand	Mileage
0	Maruti	28
2	Maruti	30

This output shows that:

- Only rows satisfying **both conditions** are displayed
- Other rows are **filtered out**

Thus,.loc[] is mainly used for **label-based filtering and condition checking**.

Locating Rows Using .iloc[]

The .iloc[] method is used for **integer position–based selection**.

This means:

- rows are selected using **index numbers**
- indexing **starts from 0**
- it does **not depend on column names**
- useful when we want **specific row positions**

Example Dataset for .iloc[]

```
import pandas as pd
```

```
lst = [  
    {"a": 5, "b": 10, "c": 15, "d": 20},  
    {"a": 25, "b": 30, "c": 35, "d": 40},  
    {"a": 45, "b": 50, "c": 55, "d": 60}  
]
```

```
d1 = pd.DataFrame(lst)  
print(d1)
```

Output Explanation

```
a  b  c  d  
0  5 10 15 20  
1 25 30 35 40  
2 45 50 55 60
```

- The dataset contains **3 rows and 4 columns**
- Row indices are **0, 1, and 2**

Different Uses of .iloc[]

Type 1: Selecting a Single Row Using Integer Index

```
d1.iloc[[0]]
```

Output

```
a b c d
0 5 10 15 20
```

Meaning:

Only the **first row** (index 0) is selected.

Type 2: Selecting Multiple Rows Using a List of Index Values

```
d1.iloc[[0, 1]]
```

Output

```
a b c d
0 5 10 15 20
1 25 30 35 40
```

Meaning:

Rows with index **0 and 1** are displayed together.

Type 3: Selecting Rows Using a Slice

```
d1.iloc[:3]
```

Output

```
a b c d
0 5 10 15 20
1 25 30 35 40
2 45 50 55 60
```

Meaning:

All rows from **index 0 up to 2** are selected.

Difference Between .loc[] and .iloc[]

Feature	.loc[]	.iloc[]
Selection type	Label-based	Integer position-based
Uses column names	Yes	No
Uses index numbers	Not required	Required
Includes last value in range	Yes	No (like Python slicing)

Describing the Dataset

To understand the dataset quickly, pandas provides useful functions.

1. describe()

Shows **statistical summary**, such as:

- mean
- standard deviation
- minimum and maximum values
- percentiles

Syntax

```
df.describe()
```

Example

```
import pandas as pd
```

```
data = {  
    "Marks": [45, 60, 72, 80, 90],  
    "StudyHours": [2, 3, 4, 5, 6]  
}
```

```
df = pd.DataFrame(data)
```

```
print(df.describe())
```

Explanation of the Code

- A **DataFrame** is created with two numerical columns: **Marks** and **StudyHours**.
- Each row represents the **data of one student**.
- The function `df.describe()` is used to compute the **statistical summary** of both columns.
- This helps in **quickly understanding the dataset without manual calculation**.

Output of describe ()

The output produced by this program will be similar to:

```
Marks StudyHours  
count    5.000000    5.000000  
mean     69.400000    4.000000
```

```
std17.520000  1.581139
min   45.000000  2.000000
25%   60.000000  3.000000
50%   72.000000  4.000000
75%   80.000000  5.000000
max   90.000000  6.000000
```

2. Dataset Shape

The **shape** tells:

- number of rows
- number of columns

df.shape

Output format:

(rows, columns)

Example:

(100, 5)

means **100 records and 5 attributes**.

These basic operations help us **understand the structure and quality of data before preprocessing begins**.

Step 3: Handling Missing Values

In datasets, some data entries may be **empty, incomplete, or undefined**. Such empty values are known as **missing values** and are usually represented as **NaN (Not a Number)** in Python.

Missing values may occur due to several reasons:

- Required information **was not collected** during data entry
- A **machine or sensor malfunctioned** while recording data
- A person **skipped a question** in a survey form
- Data was **corrupted or lost** during storage or transmission

Before performing **data analysis or machine learning**, it is essential to handle missing values because:

- Many algorithms **cannot process NaN values**
- Missing data may lead to **incorrect predictions**
- Statistical calculations may become **misleading**

Therefore, a data analyst must **choose an appropriate method** to treat missing values.

Common Methods to Handle Missing Data

1. Ignoring the Record

If only a **few rows** contain missing values, those rows can be **removed** from the dataset.

2. Filling Values Manually

The analyst may **insert a suitable value** based on real-world knowledge.

Example:

If the **age of a student** is missing, it may be filled using **known personal records**.

3. Using a Global Constant

All missing entries can be replaced with a **fixed constant value**, such as:

- 0
- -1
- "Unknown"

4. Using Mean or Median

A common statistical solution is to replace missing values using:

- **Mean** → average of all values in the column
- **Median** → middle value after sorting

5. Class-wise Mean or Median

If the dataset contains **different categories or classes**, the missing value can be replaced using the **mean of that specific class**.

6. Most Probable Value Method

Advanced approaches estimate missing values using:

- **Probability models**
- **Regression techniques**
- **Machine learning algorithms**

Program: Filling Missing Values

```
import pandas as pd
```

```
import numpy as np
```

```
data = {  
    "Age": [20, 22, np.nan, 21, np.nan],  
    "Marks": [65, 70, 75, np.nan, 80]  
}
```

```
df = pd.DataFrame(data)
```

```
print("Original Dataset with Missing Values:")
```

```
print(df)
```

Output

```
Age Marks  
0 20.0 65.0  
1 22.0 70.0  
2 NaN 75.0  
3 21.0NaN  
4 NaN 80.0
```

This output shows:

- Missing **Age values** in rows **2 and 4**
- Missing **Marks value** in row **3**

Method 1: Replace Missing Values with 0

```
print(df.fillna(0))
```

Output

```
Age Marks  
0 20.0 65.0  
1 22.0 70.0  
2 0.0 75.0  
3 21.0 0.0  
4 0.0 80.0
```

Meaning

- All **NaN values become 0**
- Easy method but **may change data meaning**

Method 2: Remove Rows with Missing Values

```
print(df.dropna())
```

Output

Age Marks

0 20.0 65.0

1 22.0 70.0

Meaning

- Only rows **without missing values remain**
- Dataset size becomes **smaller**

Method 3: Replace Missing Values with Mean

```
df["Age"].fillna(df["Age"].mean(), inplace=True)  
df["Marks"].fillna(df["Marks"].mean(), inplace=True)  
print(df)
```

Output

Age Marks

0 20.0 65.0

1 22.0 70.0

2 21.0 75.0

3 21.0 72.5

4 21.0 80.0

Meaning

- Missing **Age** replaced with **mean Age = 21**
- Missing **Marks** replaced with **mean Marks = 72.5**
- Dataset remains **complete without losing rows**

Step 4: Data Transformation

Data transformation means **changing raw numerical values into a better format** so that:

- analysis becomes easier
- algorithms perform correctly
- comparison between features becomes fair

1. Rescaling (Min-Max Scaling)

Rescaling converts all values into a **fixed range**, usually **0 to 1**.

Why needed?

Different features may have **different units**:

- Height → centimeters
- Salary → rupees

Large values may **dominate calculations**, so scaling is required.

2. Normalization

Normalization rescales each observation (row) so that its vector length becomes 1.

In L1 normalization, the sum of absolute values in each row becomes 1.

In L2 normalization, the Euclidean length (square root of sum of squares) of each row becomes 1.

Normalization is commonly used in:

- similarity measurement
- text mining
- clustering

3. Binarization

Binarization converts numbers into **only two values: 0 or 1**, based on a **threshold**.

Example:

If threshold = 5

- value > 5 → 1
- value ≤ 5 → 0

4. Standardization

Standardization transforms data so that:

- **Mean = 0**
- **Standard deviation = 1**

This is essential for:

- Logistic Regression
- Support Vector Machine
- Principal Component Analysis

Program: Data Transformation

```
import pandas as pd
import numpy as np
```

```
from sklearn import preprocessing
import scipy.stats as s

data = {'X':[2,5,8,6], 'Y':[10,3,6,1], 'Z':[20,30,-5,7]}
df2 = pd.DataFrame(data)

print("Original Dataset:")
print(df2)
```

Rescaling

```
scaler = preprocessing.MinMaxScaler()
scaled = scaler.fit_transform(df2)
print(scaled.round(2))
```

Result

All values now lie between **0 and 1**.

Normalization

```
normalized = preprocessing.normalize(df2, norm='l1')
print(normalized.round(2))
```

Result

Each row's **sum becomes 1**.

Binarization

```
binary = preprocessing.Binarizer(threshold=5).transform(df2)
print(binary)
```

Result

Values **above 5** → **1**, otherwise **0**.

Standardization

```
arr = np.array([[1., -2., 3.],
                [2., 0., 0.],
                [0., 1., -1.]])
```

```
scaled_arr = preprocessing.scale(arr)
```

```
print("Standardized Data:")
```

```
print(scaled_arr.round(2))
```

Result Meaning:

- Mean becomes **0**
- Standard deviation becomes **1**

Encoding Categorical Data

Machine learning algorithms **cannot understand text labels**. Therefore, categorical data must be **converted into numbers**.

Label Encoding

Each category is assigned a **unique integer**.

Example:

Color	Encoded
Red	0
Blue	1
Green	2

```
from sklearn.preprocessing import LabelEncoder
```

```
le = LabelEncoder()
```

```
df['Result'] = le.fit_transform(df['Result'])
```

One-Hot Encoding

Creates **separate binary columns** for each category.

Example:

City	Mumbai	Pune	Delhi
Pune	0	1	0

```
df = pd.get_dummies(df, columns=['City'])
```

This method **removes false ranking**.

Data Discretization

Data discretization converts **continuous numerical values into groups or ranges**.

Example:

Marks →

0-40 → Poor

41-60 → Average

61-80 → Good

81-100 → Excellent

This helps in:

- easy interpretation
- categorical analysis
- visualization

Equal Width Binning Program

```
import pandas as pd
```

```
import numpy as np
```

```
items = {'product':['Pen','Bag','Bottle','Book'],  
        'price':[20,150,60,80]}
```

```
df = pd.DataFrame(items)
```

```
bins = np.linspace(min(df["price"]), max(df["price"]), 4)
```

```
labels = ["Low","Medium","High"]
```

```
df["Price_Group"] = pd.cut(df["price"], bins=bins,  
                           labels=labels, include_lowest=True)
```

```
print(df)
```

Output

Original Dataset

```
product price
```

```
0    Pen    20
```

```
1    Bag   150
```

2 Bottle 60

3 Book 80

BINNED DATASET

product price Price_Group

0 Pen 20 Low

1 Bag 150 High

2 Bottle 60 Low

3 Book 80 Medium

Note: The bin assignment may vary slightly depending on how interval boundaries are calculated.

Types of Binning

1. Equal Frequency Binning

Each bin contains **same number of data points**.

2. Equal Width Binning

Each bin has **same numerical range**:

$\text{width} = (\text{maximum} - \text{minimum}) / \text{number of bins}$

This completes the **detailed explanation of missing value handling, transformation, encoding, and discretization**, which are **core steps of data preprocessing in data science**.

Lab Assignments

SET A

1. Create Own Dataset and Perform Basic Preprocessing

Dataset Name: StudentData.csv

Save the following data in Excel and store it as a .CSV file.

City	Age	Fees	Admitted
Mumbai	18	35000	Yes
Pune	19	42000	No
Nagpur	20	39000	Yes
Mumbai	21		Yes
Pune		41000	No

Nagpur	22	45000	Yes
Mumbai	23	48000	No
Pune	24	52000	Yes
Nagpur	25	56000	No
Mumbai	26	60000	Yes

2. Write a Python program to perform the following tasks:

1. Import Dataset and Perform Basic Exploration

- Display statistical description of dataset
- Display shape of dataset
- Display first three rows

2. Handle Missing Values

Replace missing values of:

- Age column → with mean age
- Fees column → with mean fees

3. Encode Categorical Data

The dataset contains two categorical columns:

- City
- Admitted

Perform:

- One-Hot Encoding on *City* column
- Label Encoding on *Admitted* column

SET B

1. Import Standard Dataset and Use Transformation Techniques

Dataset Name: diabetes.csv

This dataset contains medical diagnostic measurements of patients, such as:

- glucose level
- blood pressure
- insulin
- body mass index (BMI)
- age

The dataset is commonly used in machine learning classification problems and is available from:

<https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.data.csv>

2. Write a Program in Python to Perform the Following Tasks

1. Import Dataset from the Above Link

Load the dataset into a Pandas DataFrame so that preprocessing and analysis can be performed.

2. Rescaling

Normalize the dataset using the Minmax Scaler class so that all numerical values lie in the range 0 to 1.

3. Standardizing Data

Transform the dataset into a standard Gaussian distribution with:

- Mean = 0
- Standard Deviation = 1

4. Normalizing Data

Rescale each observation to a unit norm (length = 1) using the Normalizer class. This is useful in distance-based machine learning algorithms.

5. Binarizing Data

Use the Binarizer class to convert numerical values into binary form:

- Values greater than the threshold $\rightarrow 1$
- Values less than or equal to the threshold $\rightarrow 0$

SET C

1. Import Dataset and Perform Discretization of Continuous Data

Dataset Name: Student_Marks.csv

This dataset contains information related to students, such as:

- Student_ID
- Study_Hours
- Attendance
- Marks

The Marks column contains continuous numerical values, which will be converted into categorical performance levels using discretization.

The dataset can be accessed from the following link:

Dataset Link:

https://raw.githubusercontent.com/selva86/datasets/master/Student_Marks.csv

2. Write a Python Program to Perform the Following Tasks

1) Import Required Libraries and Load the Dataset

Write Python code to:

- import necessary libraries such as pandas and numpy
- load the dataset into a Pandas DataFrame for preprocessing and analysis

2) Display the First Five Rows of the DataFrame

Use an appropriate Pandas function to:

- preview the top five records
- understand the structure and columns of the dataset

3) Discretize the Marks Column

Convert the continuous Marks values into five discrete performance categories using the cut() function.

The five category labels should be:

- Poor
- Below_Average
- Average
- Above_Average
- Excellent

After discretization, display the first ten rows of the updated dataset.

Assignment Evaluation

0: Not Done		1: Incomplete		2: Late Complete	
3: Needs improvement		4: Complete		5: Well Done	

Signature of the Instructor

Date

Assignment 4: Data Visualization

Objectives

- To learn different statistical methods for Data visualization.
- To learn about packages matplotlib and seaborn.
- To learn functionalities and usages of Seaborn.
- Apply datavisualizationtools on various data sets.

Reading

You should read the following topics before starting this exercise:

Introduction to Exploratory Data Analysis, Data visualization and visual encoding, Data visualization libraries, Basic data visualization tools, Histograms, Bar charts/graphs, Scatter plots, Line charts, Area plots, Pie charts, Donut charts, Specialized data visualization tools, Boxplots, Bubble plots, Heat map, Dendrogram, Venn diagram, Treemap, 3Dscatter plots, Advanced data visualization tools-Wordclouds, Visualization of geospatial data, Data Visualization types.

Ready Reference

Python provides the Data Scientists with various packages both for data processing and visualization. Data visualization is concerned with the presentation of data in a pictorial or graphical form. It is one of the most important tasks in data analysis, since it enables us to see analytical results, detect outliers, and make decisions for model building. There are various packages available in Python for visualization purpose. Some of the are: matplotlib, seaborn, bokeh, plotly, ggplot, pygal, gleam, leather etc.

Steps Involved in our Visualization

1. Importing packages
2. Importing and Cleaning Data
3. Creating Visualizations

Step-1: Importing Packages

Not only for Data Visualization, but every process to be held in Python should also be started by importing the required packages. Our primary packages include Pandas for Data processing, Matplotlib for visuals, Seaborn for advanced visuals, and Numpy for scientific calculations.

Step-2: Importing and Cleaning Data

This is an important step as a perfect data is an essential need for a perfect visualization.

We have successfully imported and cleaned our dataset. Now we are set to do our visualizations using our cleaned dataset.

Step-3: Creating Visualizations

In this step we create different types of Visualizations right from basic charts to advanced charts.

There are many Python libraries for visualization, of which matplotlib, seaborn, bokeh, and ggplot are among the most popular. However, in this assignment, we mainly focus on the matplotlib library. Matplotlib produces publication-quality figures in a variety of formats, and interactive environments across Python platforms. Another advantage is that Pandas comes equipped with useful wrappers around several matplotlib plotting routines, allowing for quick and handy plotting of Series and DataFrame objects. The matplotlib library supports many more plot types that are useful for data visualization. However, our goal is to provide the basic knowledge that will help you to understand and use the library for visualizing data in the most common situations.

Self Activity

Installing Mplotlib

There are multiple ways to install the matplotlib library. The easiest way to install matplotlib is to download the Anaconda package. Matplotlib is default installed with Anaconda package and does not require any additional steps.

- Download anaconda package from the official site of Anaconda

To install matplotlib, go to anaconda prompt and run the following command

```
Pip install matplotlib
```

- Verify whether the matplotlib is properly installed using the following command in Jupyternotebook

```
import matplotlib  
matplotlib.version
```

How to use Matplotlib

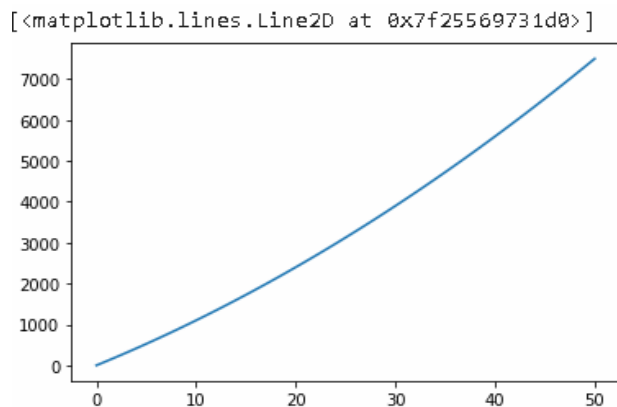
Before using matplotlib, we need to import the package. This can be done using the 'import' method in Jupyter notebook. PyPlot is the graphical module in matplotlib which is mostly used for data visualisation, importing PyPlot is sufficient to work around data visualisation.

Import matplotlib.pyplot as plt

Create a Simple Plot

Here we will be depicting a basic plot using some random numbers generated using NumPy. The simplest way to create a graph is using the 'plot()' method. To generate a basic plot, we need two axes (X) and (Y), and we will generate two random numbers using the 'linspace()' method from Numpy.

```
#import the NumPy package
import numpy as np
# generate random number using NumPy, generate two sets of random
numbers and store in x, y
x=np.linspace(0,50,100)
y=x*np.linspace(100,150,100)
# Create a basic plot
plt.plot(x,y)
```



Adding Elements to Plot

The plot generated above does not have all the elements to understand it better. Let's try to add different elements for the plot for better interpretation. The elements that could be added for the plot includes title, x- Label, y-label, x-limits, y-limits

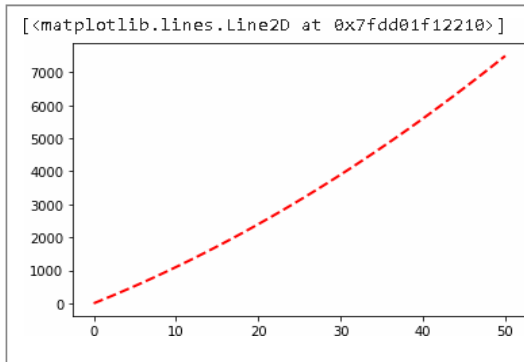
```
#set different elements to the plot generated
above # Add title using 'plt.title'
#Add x-label using
'plt.xlabel' #Add y-label
using 'plt.ylabel'
#set x-axis limits using
'plt.xlim' #set y-axis limits
```

Let's, add few more elements to the plot like colour, markers, line customization.

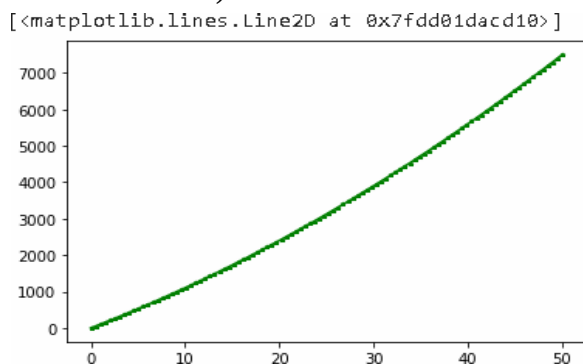
```
#add color,style,width to line element
plt.plot(x,y,c='r',linestyle='--',linewidth=2)
```

Import matplotlib as mpl

#import the pyplot module from matplotlib as plt(short name used for referring the object)

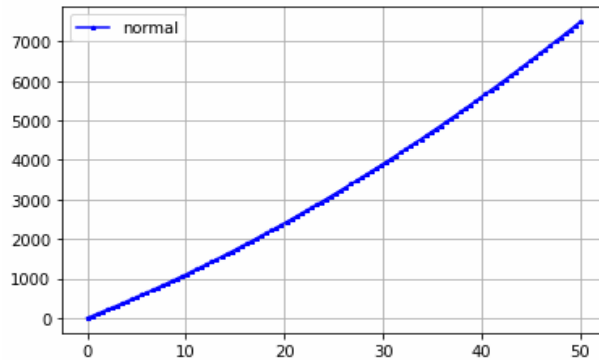


#add markers to the plot, marker has different elements i.e., style, color,sizeetc., plt.plot (x, y, marker='*', c='g', markersize=3)



Plots could be customized at three levels:

```
#add grid using grid () method
plt.plot(x,y,marker='*',markersize=3,c='b',label='normal')
plt.grid(True)
#addlegendandlabel
plt.legend()
<matplotlib.legend.Legend at 0x7fdd01c345d0>
```



• Colours

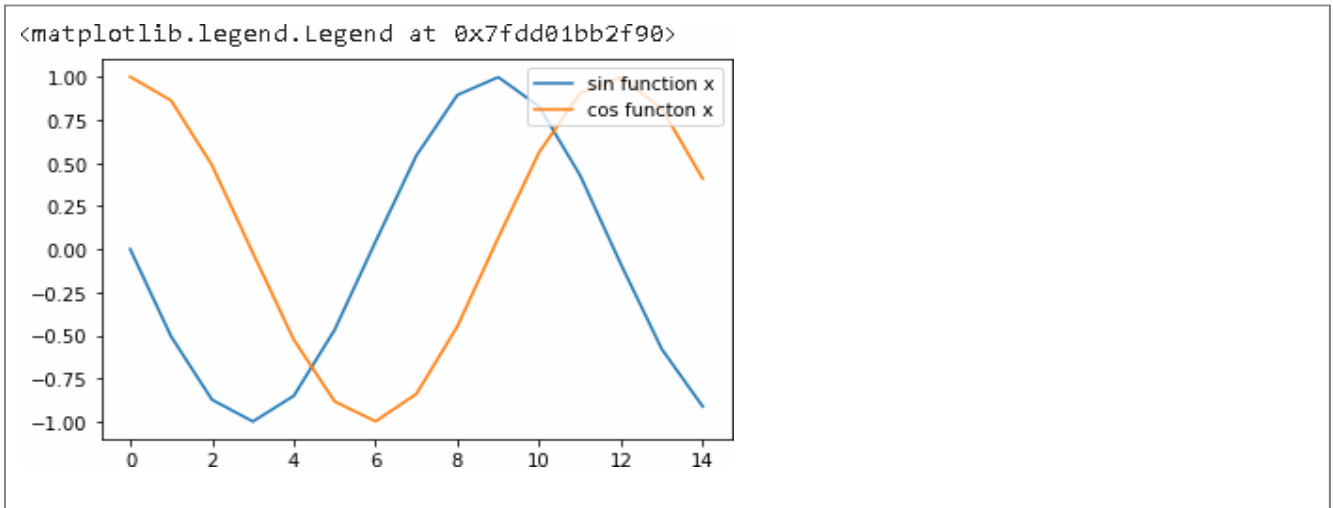
- b-blue
- k-black
- g-green
- c-cyan

Making Multiple Plots in One Figure

There could be some situations where the user may have to show multiple plots in a single figure for comparison purpose. For example, a trader wants to know the sales trend of two stores for the last 12 months and he would like to see the trend of the two stores in the same figure.

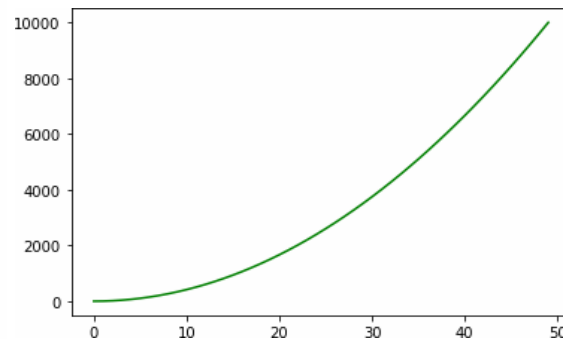
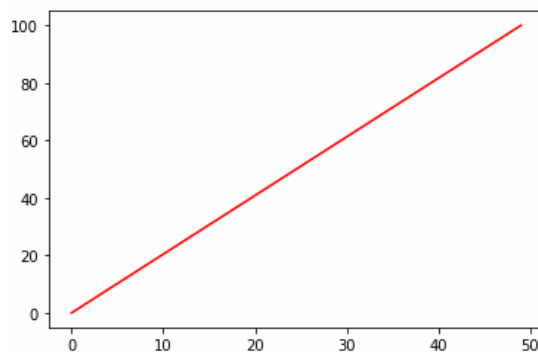
Let's plot two lines $\sin(x)$ and $\cos(x)$ in a single figure and add legend to understand which line is what.

```
#lets plot two lines Sin(x) and Cos(x)
#loc is used to set the location of the legend on the plot
#label is used to represent the label for the line in the legend
#generate the random number
x= np.arange(0,1500,100)
plt.plot(np.sin(x),label='sinfunction x')
plt.plot(np.cos(x),label='cosfunction x')
plt.legend(loc='upper right')
```



#To show the multiple plots in separate figure instead of a single figure, use plt.show() statement before the next plot statement as shown below

```
x= np.linspace (0,100,50)
plt.plot(x, 'r', label='simple x')
plt.show()
plt.plot(x*x, 'g', label='twotimesx')
plt.show()
plt.legend(loc='upperright')
```



Create Subplots

There could be some situations where we should show multiple plots in a single figure to show the complete storyline while presenting to stakeholders. This can be achieved with the use of subplot in matplotlib library. For example, a retail store has 6 stores and the manager would like to see the daily sales of all the 6 stores in a single window to compare. This can be visualised using subplots by representing the charts in rows and columns.

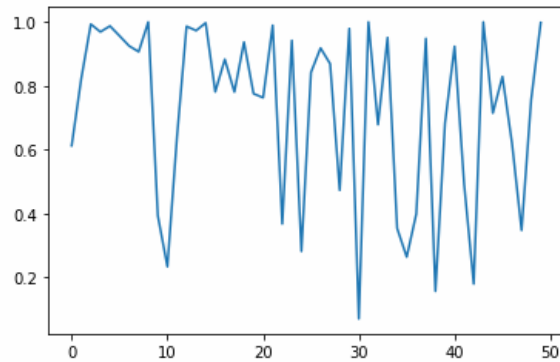
```
# sub plots are used to create multiple plots in a single
figure
# let's create a single subplot first following by adding more
subplots
x=np.random.rand(50)
```

Create Subplots

There could be some situations where we should have multiple plots in a single figure to show the complete storyline while presenting to stakeholders. This can be achieved with the use of subplot in matplotlib library. For example, a retail store has 6 stores and the manager would like to see the daily sales of all the 6 stores in a single window to compare. This can be visualised using subplots by representing the charts in rows and columns.

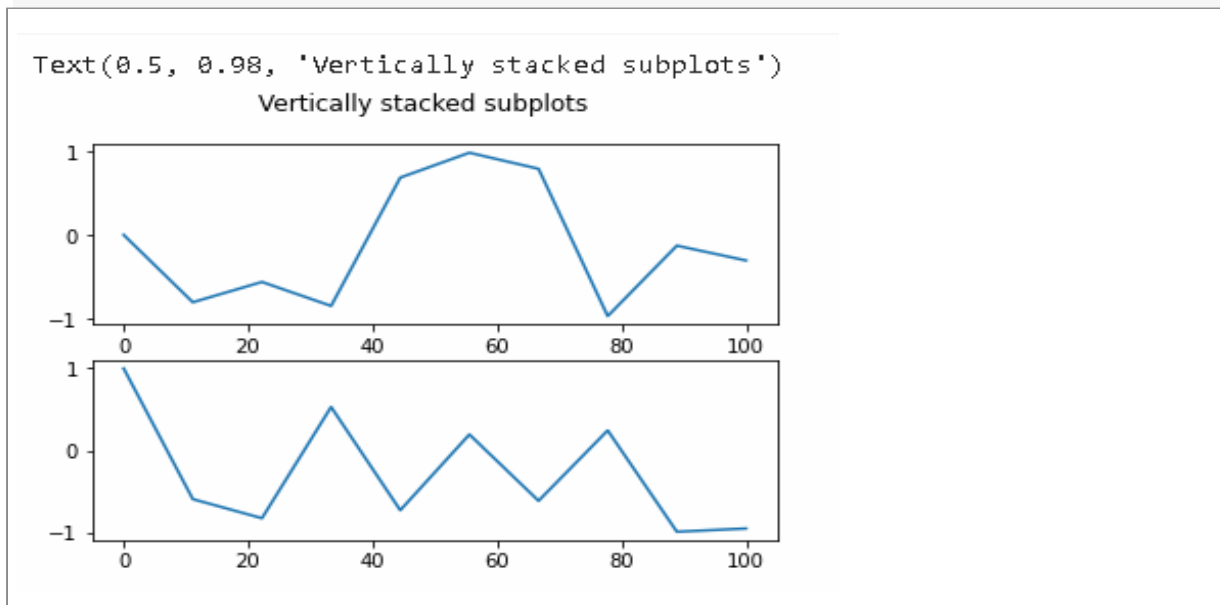
```
#need to create an empty figure with an axis as below,figure
and axis are two separate objects in matplotlib fig,
ax = plt.subplots()
#add the charts to the plot ax.plot(y)
```

```
[<matplotlib.lines.Line2D at 0x7fdd01971dd0>]
```



```
#Let's add multiple plots using subplots()function
#Give the required number of plots as an argument in subplots(), below function
creates 2 subplots
Fig,axs=plt.subplots(2) #create
data x=np.linspace(0,100,10)
#assign the data to the plot using axs axs[0].plot(x,
np.sin(x**2))
axs[1].plot(x,np.cos(x**2))
```

```
# add a title to the subplot figure fig.suptitle('Vertically stacked subplots')
```



```
#Create horizontal subplots
#Give two arguments rows and columns in the subplot()function
# subplot() gives two dimensional array with 2*2 matrix
#need to provide axal so similar 2*2 matrix as below
fig,
```

```
#add the data to the plots
ax1.plot(x, x**2) ax2.plot(x, x**3)
ax3.plot(x, np.sin(x**2))
ax4.plot(x, np.cos(x**2))
# add title fig.suptitle('Horizontal
plots')
another simple way of creating multiple sub plots as below, using axs fig,axs=plt.subplots(2, 2)
#add the data referring to row and column axs[0,0].plot(x, x**2,'g')
axs[0,1].plot(x,x**3,'r')
axs[1,0].plot(x,np.sin(x**2),'b')
axs[1,1].plot(x,np.cos(x**2),'k') # add
title
fig.suptitle('matrix subplots') #let's
create a figure object
# change the size of the figure is 'figsize = (a,b)' a is width and
'b' is height in inches
#create a figure object and name it as fig fig = plt.figure(figsize=(4,3))
#create a sample data X=np.array([1,2,3,4,5,6,8,9,10])
Y=X**2
#plot the figure plt.plot(X,Y)
```

Figure Object

Matplotlib is an object-oriented library and has objects, call uses and methods. Figure is also one of the classes from the object 'figure'. The object

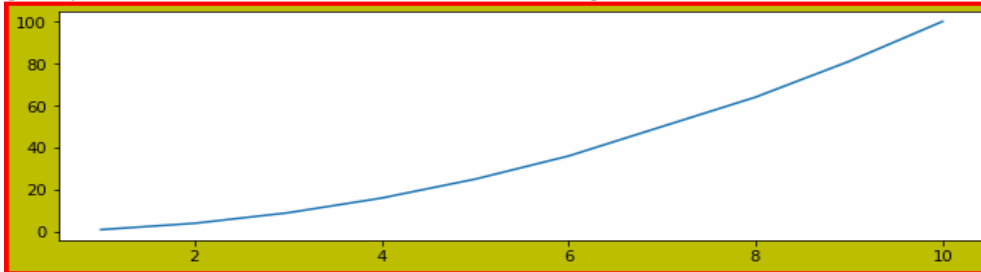
figure is a container for showing the plots and is instantiated by calling figure() function.

'plt.figure()' is used to create the empty figure object in matplotlib. Figure has the following additional parameters.

- Figsize—(width, height) in inches
- Dpi— used for dots per inch(this can be adjusted for print quality)
- facecolor
- edgecolor
- linewidth

```
#let's change the figure size and also add additional parameters like
facecolor, edgecolor, linewidth
fig =plt.figure(figsize=(10,3),facecolor='y',edgecolor='r',linewidth
=5)
#create a sample data
X=np.array([1,2,3,4,5,6,8,9,10])
Y=X**2
#plot the figure plt.plot(X,Y)
```

[<matplotlib.lines.Line2D at 0x7fdd01662390>]



Axes Object

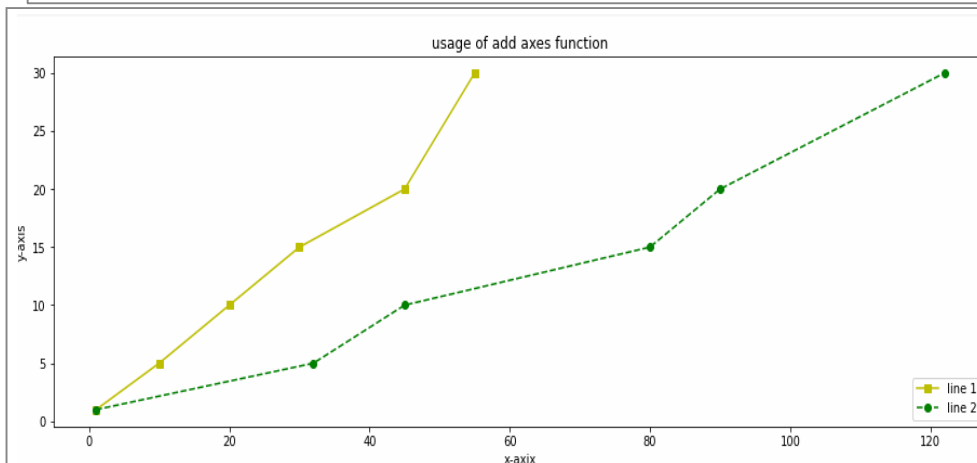
Axes is the region of the chart with data, we can add the axes to the figure using the 'add_axes()' method. This method requires the following four parameters i.e., left, bottom, width, and height

- Left – position of axes from left of figure
- bottom– position of axes from the bottom of figureList item
- width– width of the chart
- height –height of the chartList item

Other parameters that can be used for the axes object are:

- Set title using 'ax.set_title()'
- Set x-label using 'ax.set_xlabel()'
- Set y-label using 'ax.set_ylabel()'

```
#lets add axes using add_axes()method
# create a sample data
y=[1,5,10,15, 20,30]
x1=[1,10,20,30,45,55]
x2=[1,32,45,80,90,122]
#create the figure
fig = plt.figure()
# add the axes
ax= fig.add_axes([0,0,2,1])
l1=ax.plot(x1,y,'ys-')
l2=ax.plot(x2,y,'go--')
#add additional parameters
ax.legend(labels=('line1','line2'),loc='lowerright')
ax.set_title("usage of add axes function")
ax.set_xlabel('x-axis')
ax.set_ylabel('y-axis')
plt.show()
```



Different Types of Matplotlib Plots

Matplotlib has a wide variety of plot formats, few of them include bar chart, line chart, piechart, scatterchart, bubblechart, waterfallchart, circularareachart, stackedbarchartetc.,

BarGraph or Chart

Bar graph represents the data using bars either in Horizontal or Vertical directions. Bargraphs are used to show two or more values and typically the x-axis should be categorical data. The length of the bar is proportional to the counts of the categorical variable on x-axis.

Function:

- The function used to show bar graph is 'plt.bar()'
- The bar() function expects two lists of values one on x-coordinate and another on y-coordinate

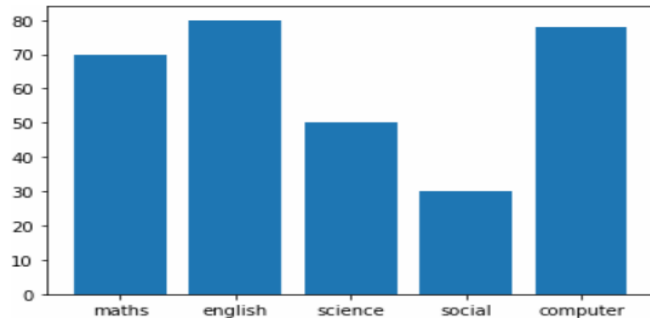
Customizations:

plt.bar() function has the following specific arguments that can be used for configuring the plot.

- Width, Color, edgecolour, linewidth, tick_label, align, bottom,

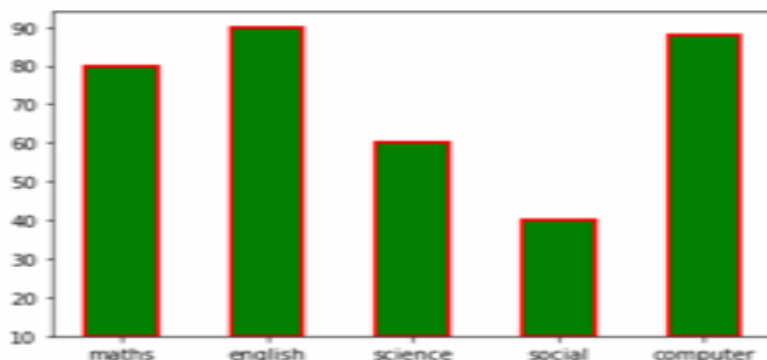
- ErrorBars-xerr,yerr

```
#lets create as imple barchart
#x-axis is shows the subject and y-axis shows the markers in
each subject
subject=['maths','english','science','social','computer'] marks
=[70,80,50,30,78]
plt.bar(subject,marks)
plt.show()
```



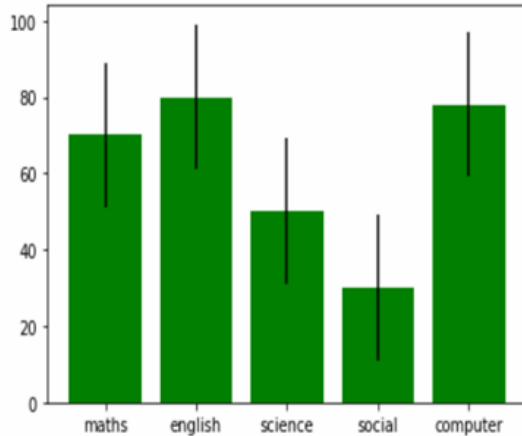
```
#let's do some customizations #width-
showsthebarwidthhanddefaultvalueis0.8 #color
- shows the bar color
#bottom-value from where they-axis starts in the chart
i.e.,the lowest value on y-axis shown
#align -move the position of x-label, has two options 'edge' or
'center'
#edge color-used to color the border so the bar #linewidth-
used to adjust the width of the line around the bar
#tick_label - to set the customized labels for the x-axis
plt.bar(subject,marks,color='g',width=0.5,bottom=10,align='cent
er',edgecolor='r',linewidth=2,
tick_label=subject)
```

<BarContainer object of 5 artists>



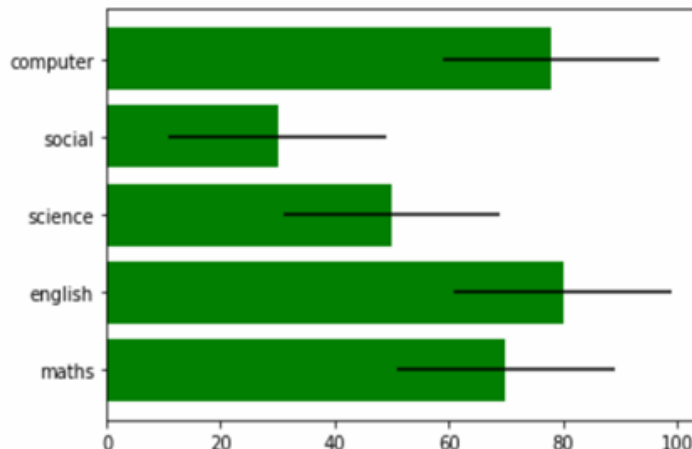
```
#errors bars could be added to represent the error values referring to an array value
# here in this example we used standard deviation to show as error bars
plt.bar(subject,marks,color ='g',yerr=np.std(marks))
```

<BarContainer object of 5 artists>



```
#to plot horizontal barplot use plt.barh() function
plt.barh(subject,marks,color ='g',xerr=np.std(marks))
```

<BarContainer object of 5 artists>



Pie Chart:

Pie charts display the proportion of each value against the total sum of values. This chart requires a single series to display. The values on the pie chart show the percentage contribution in terms of a pie called **Wedge/Widget**. The angle of the wedge/widget is calculated based on the proportion of values.

Function:

- The function used for piechart is 'plt.pie()'
- To draw a pie chart, we need only one list of values; each wedge is calculated as proportion converted into angle.

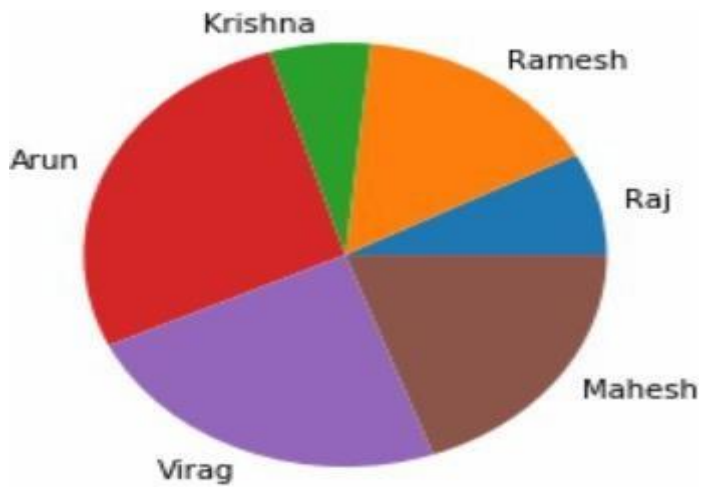
Customizations:

plt.pie() function has the following specific arguments that can be used for

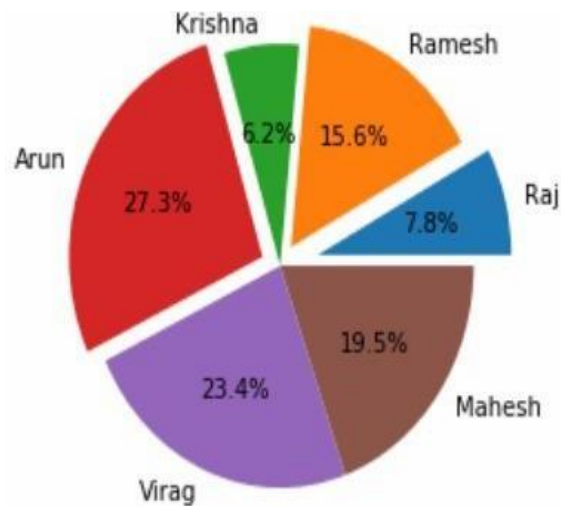
configuring the plot.

- labels–used to show the widget categories
- explode–used to pull out the widget/wedges like
- autopct–used to show the % of contributions for the widgets
- Set_aspect–used to
- shadow–to show the shadow for a slice
- colours–to set the custom colours for the wedges
- startangle–to set the angles of the wedges

```
# Let's create a simple pie plot Tickets_Closed = [10, 20, 8, 35, 30, 25]
Agents = ['Raj', 'Ramesh', 'Krishna', 'Arun', 'Virag', 'Mahesh']
plt.pie(Tickets_Closed, labels=Agents)
```



```
#Let's add additional parameters to pie plot #explode-
to move one of the wedges of the plot
#autopct - to add the contribution %
explode=[0.2,0.1,0,0.1,0,0]
plt.pie(Tickets_Closed,labels=Agents,explode=explode,
autopct='%1.1f%%' )
```



Scatter Plot

Scatter plot is used to visualize the relationship between two columns/series of data. The chart needs two variables, one variable shows X-position and the second variable shows Y-position. Scatter plot helps in understanding the following information across the two columns

- Any relationship x is between the two columns +ve Relationship
- Or-Ve relationship

Function:

- The function used for the scatter plot is 'plt.scatter()'

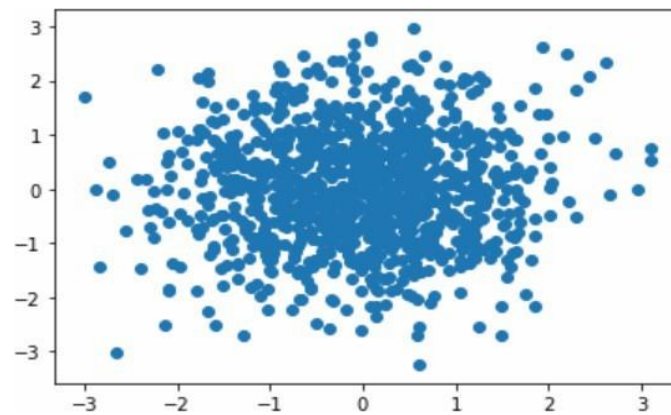
Customizations:

plt.scatter() function has the following specific arguments that can be used for configuring the plot.

- size—to manage the size of the points
- color—to set the color of the points

- marker–type of marker
- alpha–transparency of point
- norm–to normalize the data(scaling between 0 to 1)

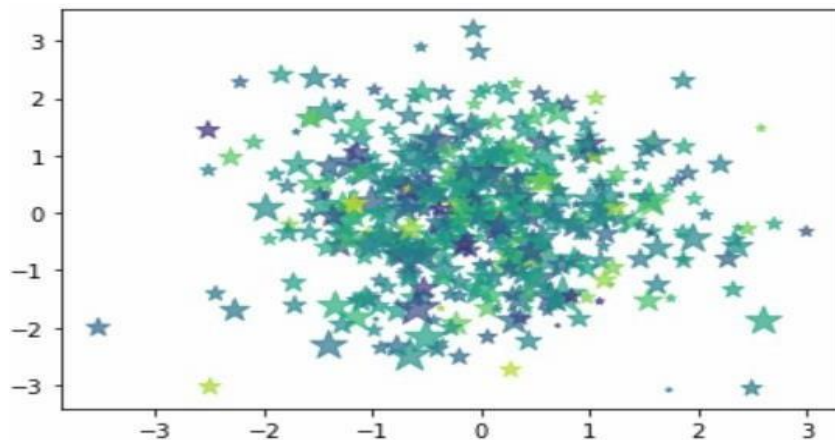
```
#generate the data with random numbers  
x = np.random.randn(1000)  
y=np.random.randn(1000)  
plt.scatter(x,y)
```



Histogram

Histogram is used to understand the distribution of the data. It is an estimate of the probability distribution of continuous data. It is similar to bar graph as discussed above but this is used to represent the distribution of a continuous variable whereas bargraph is used for discrete variable. Every distribution is characterized by four different elements including

- Center of the distribution



```
#as you observe there is no correlation exists between x and y
# let's try to add additional parameters
#size-to manage the size of the points
#color - to set the color of the points
#marker - type of marker
#alpha - transparency of point
size = 150*np.random.randn(1000)
colors=100*np.random.randn(1000)
plt.scatter(x,y,s=size,c=colors,marker='*',alpha=0.7)
```


- Spread of the distribution
- Shape of the distribution
- Peak of the distribution

Histogram requires two elements x-axis shown using bins and y-axis shown with the frequency of the values in each of the bins form the data set. Every bin has a range with minimum and maximum values.

Function:

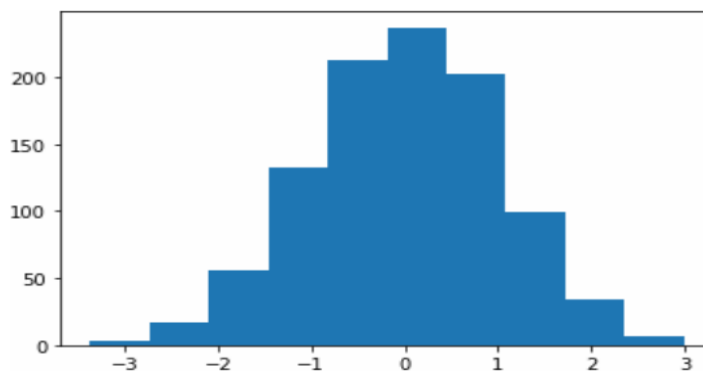
- The function used for scatterplot is 'plt.hist()'

Customizations:

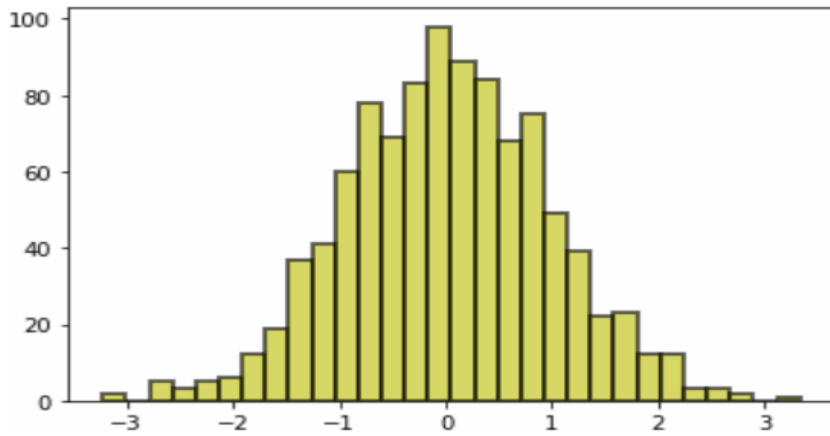
plt.hist()function has the following specific arguments that can be used for configuring the plot.

- bins–number of bins
- color
- edgecolor
- alpha–transparency of the color
- normed
- xlim–to set the x-limits
- ylim–to set the y-limits
- xticks, yticks
- facecolor, edgecolor, density

```
# let's generate random numbers and use the random numbers to
generate histogram
data=np.random.randn(1000)
plt.hist(data)
```



```
#let's add additional parameters
# facecolor
#alpha
#edgecolor
# bins
data= np.random.randn(1000)
plt.hist(data, facecolor='y', linewidth=2, edgecolor='k',
bins=30, alpha=0.6)
```



#lets create multiple histograms in a single plot # Create random data

```
hist1=np.random.normal(25,10,1000)
```

```
hist2=np.random.normal(200,5,1000)
```

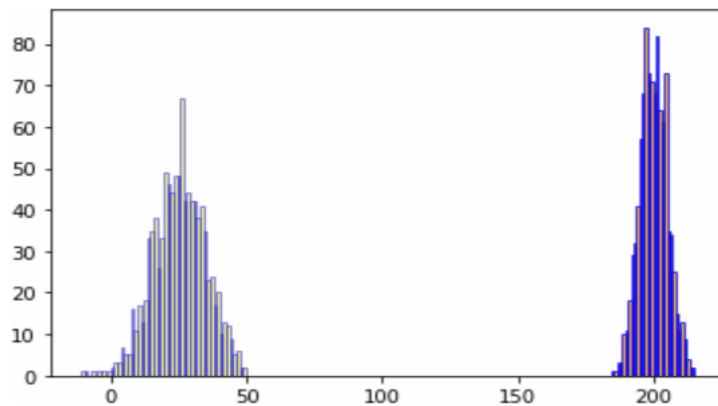
```
#plotthehistogram
```

```
plt.hist(hist1,facecolor          =      'yellow',alpha          =      0.5,          edgecolor
```

```
= 'b',bins=50)
```

```
plt.hist(hist2,facecolor          =      'orange',alpha          =      0.8,          edgecolor
```

```
= 'b',bins=30)
```



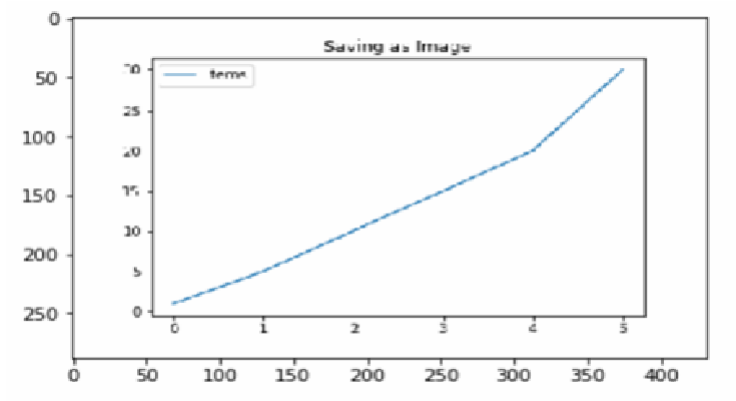
Saving Plot

Saving plot as an image using 'savefig()' function in matplotlib. The plot can be saved in multiple formats like .png, .jpeg, .pdf and many other supporting formats.

```
#let's create a figure and save it as image items =
[5,10,20,25,30,40]
x = np.arange(6)
fig=plt.figure()
ax = plt.subplot(111)
ax.plot(x,y,label='items')
plt.title('Saving as Image')
ax.legend()
fig.savefig('saveimage.png')
```

Image is saved with a file name as 'saveimage.png'.

```
#To display the image again, use the following package and commands
import matplotlib.image as mpimg
image=mpimg.imread("saveimage.png")
plt.imshow(image)
plt.show()
```



Box Plot

It displays the distribution of data based on the five-number theory by dividing the data set into three quartiles and then presents the values—minimum, maximum, median, first (lower) quartile and third (upper) quartile—in the plotted graph itself. It helps in identifying outliers and how much spread out data is from the center.

In Python, a box plot can be created using the `boxplot()` function of the `matplotlib` library. Consider the following data elements: 1, 1, 2, 2, 4, 6, 6.8, 7.2, 8, 8.3, 9, 10, 10, 11.5

Here, `min` is 1, `max` is 11.5, the lower quartile is 2, the median is 7, and the upper quartile is 9. If we plot the box plot, the code would be:

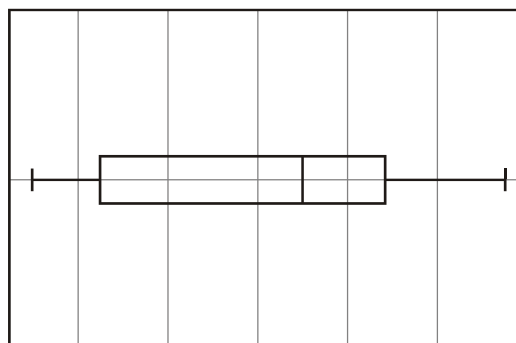
#Let us create a Simple boxplot

Import `matplotlib.pyplot` as `plt`

`data=[1,1,2,2,4,6,6.8,7.2,8,8.3,9,10,10,11.5]`

`plt.boxplot(data, vert=False) plt.show()`

The box plot appears as:



Lab Assignments

Student must use Iris flower data set for Lab Assignment

The Iris flower data set or Fisher's Iris data set is a multivariate data set introduced by the British statistician and biologist Ronald Fisher in his 1936 paper



Iris setosa



Iris versicolor



Iris virginica

The data set consists of 50 samples from each of three species of Iris (Iris setosa, Iris virginica and Iris versicolor). Four features were measured from each sample: the length and the width of the sepals and petals, in centimeters. Based on the combination of these four features, Fisher developed a linear discriminant model to distinguish the species from each other.

Set A

1. Generate a random array of 50 integers and display them using a linechart, scatter plot, histogram and box plot. Apply appropriate color, labels and styling options.
2. Add two outliers to the above data and display the box plot.
3. Create two lists, one representing subject names and the other representing marks obtained in those subjects. Display the data in a pie chart and bar chart.
4. Write a Python program to create a Bar plot to get the frequency of the three species of the Iris data.
5. Write a Python program to create a Pie plot to get the frequency of the

three species of the Iris data.

6. Write a Python program to create a histogram of the three species of the Iris data.

SET B

1. Write a Python program to create a graph to find relationship between the petal length and petal width.
2. Write a Python program to draw scatterplots to compare two features of their is dataset.
3. Write a Python program to create box plots to see how each feature i.e. Sepal Length, Sepal Width, Petal Length, Petal Width are distributed across the three species.

SET C

1. Write a Python program to create a pair plot of there is dataset and check which flower species seems to be the most separable.
2. Write a Python program to generate a box plot to show the Interquartile range and outliers for the three species for each feature.
3. Write a Python program to create a join plot using "kde" to describe individual distributions on the same plot between Sepal length and Sepal width.

Note: The kernel density estimation (kde) procedure visualizes a bivariate distribution. In seaborn, this kind of plot is shown with a contour plot and is available as a style in joint plot().

Assignment Evaluation

0: Not Done		1: Incomplete		2: Late Complete	
3: Needs improvement		4: Complete		5: Well Done	

Signature of the Instructor

Date

